Principia Softwarica: The ARM Emulator `5i`

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
???

March 12, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **6**
- 1.1 Motivations . . . 6
- 1.2 The Plan 9 ARM emulator: `5i` . . . 6
- 1.3 Other emulators . . . 7
- 1.4 Getting started . . . 8
- 1.5 Requirements . . . 9
- 1.6 About this document . . . 9
- 1.7 Copyright . . . 9
- 1.8 Acknowledgments . . . 9

2 Overview **10**
- 2.1 Computer principles . . . 10
  - 2.1.1 Von Newmann architecture . . . 10
  - 2.1.2 Instruction format . . . 10
  - 2.1.3 Hardware versus software . . . 10
  - 2.1.4 IO . . . 10
  - 2.1.5 Interrupts . . . 10
  - 2.1.6 Virtual memory . . . 10
  - 2.1.7 Endianess . . . 10
- 2.2 Emulator principles . . . 10
- 2.3 `5i` interfaces . . . 10
  - 2.3.1 Command-line interface . . . 10
  - 2.3.2 Debugger interface . . . 10
- 2.4 `helloworld` . . . 11
- 2.5 Input binary executable . . . 11
- 2.6 The ARM architecture . . . 11
- 2.7 Code organization . . . 12
- 2.8 Software architecture . . . 12
- 2.9 Book structure . . . 13

3 Core Data Structures **14**
- 3.1 `Instruction` and `Opcode` . . . 14
- 3.2 `Inst` and `itab` . . . 15
- 3.3 `Registers` and `reg` . . . 15
- 3.4 `Segment` and `memory` . . . 15

4 `main()` **17**
- 4.1 `inithdr()` . . . 18
- 4.2 `initmemory()` . . . 18
- 4.3 `initstk()` . . . 20

4.4 cmd() . . . 21

5 Instruction Interpreter **23**
5.1 Instruction binary format . . . 23
5.2 run() . . . 23
5.3 arm_class() . . . 24
5.4 Arithmetic and logic . . . 24
5.4.1 Opcode extraction . . . 24
5.4.2 Multiplication . . . 25
5.4.3 Idp() and dpex() . . . 26
5.4.4 Boolean logic . . . 28
5.4.5 S bit . . . 28
5.4.6 Addition and overflow . . . 28
5.4.7 shift() . . . 29
5.4.8 Immediate values . . . 32
5.4.9 64 bits target (signed/unsigned) multiplication . . . 33
5.4.10 Misc instructions . . . 34
5.5 Memory . . . 34
5.5.1 Opcode extraction . . . 34
5.5.2 Memory swap . . . 35
5.5.3 Load/store . . . 36
5.5.4 Multi registers load/store . . . 39
5.6 Control flow . . . 40
5.6.1 Opcode extraction . . . 40
5.6.2 Simple branching . . . 41
5.6.3 Branch and link . . . 41
5.6.4 Comparisons . . . 41
5.6.5 ARM Conditional execution . . . 42
5.7 Software interrupt . . . 44
5.8 Unimplemented instructions . . . 44

6 Memory **45**
6.1 page_of_vaddr() . . . 45
6.2 Page faults . . . 46
6.3 Tlb . . . 46
6.4 ifetch() . . . 47
6.5 The instruction cache . . . 48
6.6 getmem_xxx() . . . 48
6.7 putmem_xxx() . . . 50

7 Syscalls Emulation **52**
7.1 memio() . . . 53
7.2 Nop and errstr syscalls . . . 54
7.3 Memory syscalls . . . 55
7.4 File and IO syscalls . . . 55
7.5 Directory syscalls . . . 59
7.6 Namespace syscalls . . . 61
7.7 Misc syscalls . . . 61
7.8 Unsupported syscalls . . . 63

8 Debugger **67**
8.1 Overview . . . 67
8.2 Interface . . . 68
8.2.1 Inspecting: `$` . . . 71
8.2.2 Controlling: `:` . . . 73
8.3 Format . . . 74
8.4 Dumpers . . . 78
8.5 Traces . . . 79
8.5.1 Syscalls trace . . . 79
8.5.2 Stack trace . . . 79
8.5.3 Call tree trace . . . 81
8.6 Breakpoints . . . 81
8.6.1 Code breakpoint . . . 84
8.6.2 Memory breakpoint . . . 84

9 Profiler **85**

10 Advanced Topics **90**
10.1 System instructions . . . 90
10.2 Coprocessor instructions . . . 90
10.3 Float instructions . . . 90
10.4 Signals . . . 90
10.5 Optimisations . . . 90

11 Conclusion **91**

A Debugging **92**

B Error Management **93**

C Utilities **94**
C.1 Memory Management . . . 94

D Extra Code **95**
D.1 `machine/5i/` . . . 95
D.1.1 `machine/5i/arm.h` . . . 95
D.1.2 `machine/5i/icache.c` . . . 97
D.1.3 `machine/5i/globals.c` . . . 97
D.1.4 `machine/5i/utils.c` . . . 98
D.1.5 `machine/5i/bpt.c` . . . 98
D.1.6 `machine/5i/mem.c` . . . 99
D.1.7 `machine/5i/symbols.c` . . . 99
D.1.8 `machine/5i/run.c` . . . 99
D.1.9 `machine/5i/5i.c` . . . 101
D.1.10 `machine/5i/stats.c` . . . 101
D.1.11 `machine/5i/cmd.c` . . . 102
D.1.12 `machine/5i/syscall.c` . . . 102

Glossary **105**

Index **106**

Chapter 1

Introduction

The goal of this book is to present with full details the source code of a processor emulator.

1.1 Motivations

Why a processor emulator? Because I think you are a better programmer if you fully understand how things work under the hood, and the processor is really at the bottom of what is under the hood.

Every other books in Principia Softwarica will describe programs that ultimately runs on a concrete machine. For the assembler and linker, I will even describe programs that generate binary codes for a specific architecture (the ARM). It is thus useful, especially to understand the assembler, linker, and also compiler, to have somewhere the full description of the instruction set of an architecture (ISA): its binary format, mnemonic names, and also semantics. In fact, the hardware can also be seen as a kind of software: a processor is an interpreter for a low-level language, and so it is also *softwarica* material. The binary format of some instructions and their semantics can be described simply by explaining the code of an interpreter: a processor emulator.

Here are a few questions I hope this book will answer:

- What is the list of all processor instructions? What can a typical computer do?
- What are the most important instructions? How can they be used to implement higher level constructs?
- Which instruction allows to enter in kernel mode?
- How instructions are encoded? What is the binary format?
- How is handled overflow? What are the differences between the logic shift-right and arithmetic shift-right operations?
- How does an ARM machine compare to a Turing machine? To a Von Newman machine?

1.2 The Plan 9 ARM emulator: `5i`

I will not describe in this book a processor in the form of a program of an hardware description language (e.g., VHDL), which operates at the granularity of logic gates (e.g., `or`, `and`, `nand`). For such an approach I recommend the great book *The Elements of Computing Systems* [NS05] and its companion website `http://www.nand2tetris.org`. Instead, I will present the source code of an *emulator* written in the high-level language C, which allows us to describe a relatively complex processor and its machine language in a book of a reasonable size. An emulator can be viewed as an executable specification[1] of a machine.

[1] This program will be useful also for testing purpose when I will describe in other books the assembler, linker, and compiler.

I will explain in this book the code of the ARM emulator `5i`[2] which is about 4400 lines of code (LOC). `5i` emulates the execution of an ARM binary in a Plan 9 environment. The `5` comes from the Plan 9 convention to name architecture with a number or single letter (0 is MIPS, 5 is ARM, 8 is x86, etc), and the `i` probably means interpreter.

I chose the ARM architecture over the x86 architecture because even if x86 is the processor of most desktop machines today, ARM has a far simpler architecture. Indeed, RISC (Reduced Instruction Set Computer) machines such as an ARM, as their name suggest, have a smaller set of instructions than CISC (Complex Instruction Set Computer) machines such as an x86, and have also simpler instructions. The code of an ARM emulator is thus smaller and simpler to describe, while still conveying in my opinion the essence of all processors.

I chose ARM over MIPS, because even if the MIPS is a RISC machine probably simpler than the ARM, there is not much remaining MIPS machines around. The ARM on the other hand is very much alive; it is the most popular processor in phones today. It is also the processor of the extremely cheap Raspberry Pi[3] machine, a machine used by many electronic hobbyist. This also makes ARM a great candidate for our teaching purpose.

Note that `5i` is not a complete processor simulator though. It emulates all the basic instructions, and so is a good reference for the ARM ISA, but it does not emulate low-level things such as hardware interrupts, device interactions (IO), coprocessor instructions, kernel and user modes, or the booting process. The only system instruction partially handled is the software interrupt, and it is emulated by mimicing the semantics of the Plan 9 system calls[4]. `5i` can run simple Plan 9 programs compiled for the ARM, but it can not run a kernel. However, I think `5i` is a good compromise for our educational purpose.

Note also that describing a C program, and not a VHDL program, has a few disadvantages. Indeed, to emulate for instance the `AND` ARM instruction, I will show C code such as:

⟨*instruction interpreter simplified snippet* 7⟩≡

```
case AND_INSTRUCTION:
  register3 = register1 & register2;
  ...
  break
```

This code uses the C binary `&` operator. I am cheating in some sense, just like when someone describes the code of a Lisp interpreter written in Lisp; it does not help to understand fully how Lisp is implemented, for instance, to understand how a garbage collector works. In our case, using C does not help to understand how certain operations work at the bit level, how for instance a 32 bits adder, or multiplier, or multiplexer works[5]. Is is still useful though to get an overview of the fundamental capabilities and features offered by most computers and to better understand the ARM ISA.

1.3 Other emulators

Here are a few emulators that I considered for this book, but which I ultimately discarded:

- SPIM[6] is a MIPS processor emulator. It is one of the first emulator written for educational purposes. It is partially described in an appendix of the classic architecture book *Computer Organization and Design: The Hardware/Software Interface* [PH13]. However, as I mentioned it earlier, MIPS is unfortunately a dying machine. Moreover, SPIM does not execute *binary* MIPS programs; instead, it interprets *assembly* MIPS programs. This hybrid approach would then force me to describe the SPIM assembly scanner and parser, instead of focusing on the binary format of instructions, which is in my opinion an important thing

[2] http://plan9.bell-labs.com/magic/man2html/1/vi, which despite its name covers also `5i`

[3] https://www.raspberrypi.org/

[4] As a side effect, this book also helps to understand the kernel, especially its API and the semantic of a few important system calls such as `sysbrk()`.

[5] Again I recommend [NS05] for that.

[6] http://pages.cs.wisc.edu/~larus/spim.html

to understand for Principia Softwarica. Finally, the 15 000 LOC of SPIM are bigger than the 4200 LOC of `5i`.

- QEMU[7] is a popular and fast emulator supporting many architectures, including ARM and x86. Instead of the simple interpretation approach used by `5i`, QEMU uses dynamic binary translation, which improves a lot its performance. QEMU emulates everything (system instructions, faults, devices, etc), and so can run entire kernels. In fact, I suggest you to use QEMU to experiment with Plan 9. However, the codebase of QEMU is very large: more than 1.2 million LOC in total. Even its `hw/arm` subdirectory is already more than 25 000 LOC.

- MAME[8] is a popular emulator for arcade machines (e.g., R-type, Pong), consoles (e.g., Atari 2600, Nes, Gameboy, NeoGeo), calculators (e.g., ti85), and vintage computers (e.g., Alto, z80, Atari ST, Amiga). It is used mainly to play old video games (programmed for old machines) on modern computers. It supports hundreds of such machines and can run thousands of video games. But this generality has a price; its codebase is very big, almost 5 millions LOC. It would be too hard to describe this emulator in a book, even if I would focus just on one specific machine.

- Hack[9] is a very simple machine described in [NS05] and used for educational purpose. Its emulator consists of 7000 LOC. It also comes with a very nice debugger of 6000 LOC. Hack is a great resource to learn about architecture and to understand how a simple processor works. However, the processor is too restricted and arguably too simple for Principia Softwarica. Moreover, the Hack machine does not exist for real, and programmers have written very few tools for it. For instance, there is neither a C compiler targeting this architecture nor a real operating system for it.

- MMIX[10] and its ancestor MIX are computers designed by Donald Knuth and used in his classic book series *The Art of Computer Programming* [**?**]. Donald Knuth also wrote A book using literate programming explaining the full code of an MMIX emulator [Knu99] (including a description of the processor pipeline, the floating point unit, tje assembler). Howoever, similar to Hack, there are very programs for this machine. For instance, Donald Knuth in his books assumes the presence of an operating system called NNIX, but nobody has ever written it.

1.4 Getting started

To play with `5i`, you will first need to install the Plan 9 fork used in Principia Softwarica. See https://www.principia-softwarica.org/. Once installed, you can test `5i` under Plan 9 with:

```
[1] $ cd /tests/5a/
[2] $ 5a helloa.s
[3] $ 5l helloa.5 -o helloa
[4] $ 5i helloa
[5] 5i> :c
[6] hello world
[7] exists(end)
[8] stopped at #1068 _main+48
[9] $
```

[7] http://www.qemu.org
[8] http://www.mame.net/
[9] http://www.nand2tetris.org/05.php
[10] http://www-cs-faculty.stanford.edu/~uno/mmix.html

The commands in lines 2 and 3 respectively assembles and links the very simple `helloa.s` hello-world ARM assembly program (or cross-assemble and cross-link if you are on a non-ARM host machine). Line 4 runs the emulator on the ARM binary program `helloa`. `5i` has an interface similar to a debugger, and so the input command `:c` entered after the `5i>` prompt is used to tell the emulator to `c`ontinue the execution of the emulated program `helloa`. `5i` then interprets the program, which outputs `hello world` at line 6. `5i` then outputs a few debugging information at lines 7 and 8 before exiting.

1.5 Requirements

Because most of this book is made of C source code, you will need a good knowledge of the C programming language [KR88] to understand it.

Note also that this book is not an introduction to computer architecture. I assume you already have a basic understanding of how a processor works, and so that you are familiar with concepts such as registers, memory addressing modes, interrupts, etc. I assume you already know most of the theory; this book is here to cover the practice. See [Tan88, PH13] instead for excellent introductions to computer architecture.

It is not necessary to know the ARM architecture to understand this book. In fact, this book can be used as an introduction to the ARM processor. However, I highly recommend to print the excellent colorful ARM reference card `http://re-eject.gbadev.org/files/armref.pdf`. It will help you to visually understand the binary format of the ARM instructions. This card is especially helpful to understand the code that does many bit manipulations to generate the different parts an ARM instruction.

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the author of `5i`, who wrote in some sense most of this book.

Chapter 2

Overview

2.1 Computer principles

2.1.1 Von Newmann architecture

2.1.2 Instruction format

2.1.3 Hardware versus software

2.1.4 IO

2.1.5 Interrupts

2.1.6 Virtual memory

2.1.7 Endianess

2.2 Emulator principles

2.3 `5i` interfaces

2.3.1 Command-line interface

2.3.2 Debugger interface

```
% actually provides a debugger interface a la 'db'
%
% so 5i 5.out, then at prompt type   ":c"
% to continue (that is run the program)
%
% also good to do $t :c
% so have a trace with all the instructions!! cool to debug
%
% $r to dump the registers
%
%How when get a pagefault display the current instruction?
%
%$q to exit
%
```

```
%? vs /
%main ?i
%exit ?iii
%
%can do
%$t to get trace of instruction (very useful)
%:s to step
%and then just RET to repeat the last cmd
```

2.4 helloworld

2.5 Input binary executable

```
 a.out (unix/plan9) header, and then ARM.

 for a compiled helloworld.c, there are actually lots
 of instructions involved, print is complicated, _main vs main,
 and probably other stuff. Could be good to have a
 simplified helloworld.s first where do very little.


TODO: have a disassembler? could be interesting to show
 some example of binary and what Assembly it corresponds too.
 libmach does that no?

TODO: could also show some assembly and show the binary
 For instance can see AB that does the +8 implicitly.
 so if want to jump to next instruction there will be actually
 a -4 in the value of AB.
```

2.6 The ARM architecture

```
ARM ISA format
see ARM wikipedia page!
endianess: actually bi-endian, but little endian by default,
 hence va[0] | va[1] >> 8, etc

 a few weird stuff:
  - conditional execution format to compensate for lack of branch
    prediction, see wikipedia euclid example
  - no CALL/RET? do via B or BL (could also do ADD PC I think)

 lots of symetries and repetitions, see instruction format section

 raspberry = ARMv6?
```

```
section{C}
 refresher on bit manip?
 w&1< <7, x& ~(BYPG-1), 31< <7, x> >25 & 0xf, etc

http://blogs.msdn.com/b/ntdebugging/archive/2013/11/22/understanding-arm-assembly-part-1.aspx

 ex of assembly? how put 1 into register1?
```

2.7 Code organization

⟨*basic includes* 12⟩≡ (102 101 99 98 97)

```
  #include <u.h>
  #include <libc.h>
  #include <bio.h>
  #include <mach.h>

  #include "arm.h"

arm.h: DS, forward decl (a dat.h + fns.h)

 mach.h for very little I think, could probably remove it of most files

utilities:
utils.c:
globals.c:

hardware:
mem.c:
icache.c: instruction cache (nop)
syscall.c: supported syscalls emulation (just proxy)

debugger stuff:
bpt.c: breakpoints
symbols.c:
cmd.c:
stats.c:

main stuff:
run.c
5i.c
```

2.8 Software architecture

```
 Use libmach a bit: crackhdr(), symbol map.
 But archi is so simple that no need parsing: just read ulong (instruction)
 so 4 bytes from the file.
 Then decompose instruction and dispatch.
```

```
I think some of the algo like dpex(), arm_class, really emulate
the algorithm of the machine itself. The code is factorized.
it's not just a giant switch for each opcode, things in the ARM
are better structured, with some repetition.

note that no deps to 5.out.h here, it's the final binary format,
not the plan9 object ARM format.


machine language, binary format of instruction,
opcode, operands, semantic. interpreter.
```

2.9 Book structure

Chapter 3

Core Data Structures

3.1 Instruction and Opcode

The ARM uses fixed-length instructions of 4 bytes:

⟨*typedef instruction* 14a⟩≡ (95)

```
typedef ulong instruction;
```

An ARM instruction is composed of an opcode and a few operands.

⟨*enum* opcode 14b⟩≡ (95)

```
enum opcode {
    // ----------------------------------------
    // Arithmetic and logic opcodes
    // ----------------------------------------
    ⟨arith/logic opcodes 24b⟩
    // ----------------------------------------
    // Memory MOV opcodes
    // ----------------------------------------
    ⟨memory opcodes 34e⟩
    // ----------------------------------------
    // Control flow opcodes
    // ----------------------------------------
    ⟨branching opcodes 40a⟩
    // ----------------------------------------
    // Syscall opcodes
    // ----------------------------------------
    ⟨syscall opcodes 44a⟩

    // for opcodes not handled by 5i
    OUNDEF   = 89
};
```

⟨*enum* ixxx 14c⟩≡ (95)

```
enum opcode_category
{
    Iarith,
    Imem,
    Ibranch,
    Isyscall,
    Imisc,
};
```

3.2 Inst and itab

⟨*struct* Inst 15a⟩≡ (95)

```
struct Inst
{
    void  (*func)(instruction);
    char* name;
    // enum<opcode_category>
    int type;

    ⟨Inst profiling fields 85a⟩
};
```

⟨*global* itab 15b⟩≡ (99c)

```
//map<enum<opcode>, Inst>
Inst itab[] =
{
  ⟨itab elements 15c⟩
  { 0 }
};
```

Uses itab 15b.

⟨itab *elements* 15c⟩≡ (15b) 25g ▷

```
[OUNDEF] =  { &undef,  "UNDEF", Imisc},
```

Uses Imisc 14c, OUNDEF 14b, and undef() 93d.

3.3 Registers and reg

⟨*struct* Registers 15d⟩≡ (95)

```
struct Registers
{
    long r[16];
    ⟨Registers other fields 23b⟩
};
```

⟨*global* reg 15e⟩≡ (97b)

```
Registers reg;
```

⟨*enum* regxxx 15f⟩≡ (95)

```
enum
{
    REGARG = 0,
    REGRET = 0,

    REGSP = 13,
    REGLINK = 14,
    REGPC = 15,
};
```

3.4 Segment and memory

⟨*enum* segment_kind 15g⟩≡ (95)

```
enum segment_kind
{
    Text,
    Data,
```

```
    Bss,
    Stack,

    Nseg,
};
```

⟨*struct* Memory 16a⟩≡ (95)

```
struct Memory
{
    //map<enum<segment_kind>, Segment>
    Segment seg[Nseg];
};
```

Uses Nseg 15g.

⟨*global* memory 16b⟩≡ (97b)

```
Memory memory;
```

⟨*struct* Segment 16c⟩≡ (95)

```
struct Segment
{
    // enum<segment_kind>
    short type;

    uintptr base;
    uintptr end;

    //array<option<array_4096<byte>>> page table
    byte** table; // the data

    // for the Text and Data segments the bytes are in the file
    ulong fileoff;
    ulong fileend;

    ⟨Segment profiling fields 87c⟩
};
```

Chapter 4

main()

⟨*global* file 17a⟩≡ (101a)

```
char* file = "5.out";
```

Uses file 17a.

⟨*global* text 17b⟩≡ (97b)

```
fdt text;
```

⟨*global* bxxx 17c⟩≡ (101a)

```
Biobuf bi, bo;
```

⟨*global* bixxx 17d⟩≡ (97b)

```
Biobuf *bin, *bout;
```

⟨*function* main 17e⟩≡ (101a)

```
//@Scheck: entry point!
void main(int argc, char **argv)
{

    argc--;
    argv++;

    bout = &bo;
    bin = &bi;
    Binit(bout, STDOUT, OWRITE);
    Binit(bin, STDIN, OREAD);

    ⟨main() tlb initialisation 47a⟩

    if(argc)
        file = argv[0];
    argc--;
    argv++;

    text = open(file, OREAD);
    if(text < 0)
        fatal(true, "open text '%s'", file);

    Bprint(bout, "5i\n");

    inithdr(text);
    initmemory();
    initstk(argc, argv);

    cmd();
}
```

Uses OREAD, OWRITE, STDIN, STDOUT, bi 17c, bin 17d, bo 17c, bout 17d, cmd() 21c, fatal() 93a, file 17a, inithdr() 18b, initmemory() 19a, initstk() 20e, open(), text 17b, and true.

4.1 inithdr()

⟨*global* fhdr 18a⟩≡ (101a)
```
Fhdr fhdr;
```

⟨*function* inithdr 18b⟩≡ (101a)
```
void
inithdr(fdt fd)
{
    ⟨inithdr() locals 18e⟩

    seek(fd, 0, SEEK__START);
    if (!crackhdr(fd, &fhdr))
        fatal(false, "read text header");

    if(fhdr.type != FARM )
        fatal(false, "bad magic number: %d %d", fhdr.type, FARM);

    ⟨inithdr() symmap initialisation 18d⟩
    ⟨inithdr() mach initialisation 18f⟩

}
```
Uses FARM, SEEK__START, false, fatal() 93a, fhdr 18a, and seek().

⟨*global* symmap 18c⟩≡ (97b)
```
Map *symmap;
```

⟨inithdr() *symmap initialisation* 18d⟩≡ (18b)
```
if (syminit(fd, &fhdr) < 0)
    fatal(false, "%r\n");

symmap = loadmap(symmap, fd, &fhdr);
```
Uses false, fatal() 93a, fhdr 18a, and symmap 18c.

⟨inithdr() *locals* 18e⟩≡ (18b)
```
Symbol s;
// from libmach.a
extern Machdata armmach;
```

⟨inithdr() *mach initialisation* 18f⟩≡ (18b)
```
//???
if (mach->sbreg && lookup(0, mach->sbreg, &s))
    mach->sb = s.value;
machdata = &armmach;
```

4.2 initmemory()

⟨*enum* _anon_ (machine/5i/arm.h)7 18g⟩≡ (95)
```
enum
{
    /* Plan9 Kernel constants */
    BY2PG  = 4096,
    BY2WD  = 4,
```

```
    UTZERO  = 0x1000,
    STACKTOP = 0x80000000,
    STACKSIZE = 0x10000,

    ⟨constant PROFGRAN 85e⟩
    ⟨constant Sbit 28c⟩
    ⟨constant SIGNBIT 43b⟩
};
```

⟨*function* initmemory 19a⟩≡ (101a)

```
void
initmemory(void)
{
    uintptr t, d, b, bssend;
    ⟨initmemory() locals 19b⟩

    t = (fhdr.txtaddr+fhdr.txtsz+(BY2PG-1)) & ~(BY2PG-1);
    d = (t + fhdr.datsz + (BY2PG-1)) & ~(BY2PG-1);
    bssend = t + fhdr.datsz + fhdr.bsssz;
    b = (bssend + (BY2PG-1)) & ~(BY2PG-1);

    ⟨initmemory() Text segment initilisation 19c⟩
    ⟨initmemory() Data segment initilisation 19f⟩
    ⟨initmemory() Bss segment initilisation 20c⟩
    ⟨initmemory() Stack segment initilisation 20d⟩

    reg.r[REGPC] = fhdr.entry;
}
```

Uses BY2PG 18g, REGPC 15f, fhdr 18a, and reg 15e.

⟨initmemory() *locals* 19b⟩≡ (19a)

```
Segment *s;
```

⟨initmemory() *Text segment initilisation* 19c⟩≡ (19a) 19e ▷

```
s = &memory.seg[Text];
s->type = Text;
s->base = fhdr.txtaddr - fhdr.hdrsz;
s->end = t;
s->fileoff = fhdr.txtoff - fhdr.hdrsz;
s->fileend = s->fileoff + fhdr.txtsz;
s->table = emalloc(((s->end - s->base)/BY2PG) * sizeof(byte*));
```

Uses BY2PG 18g, Text 15g, emalloc() 94a, fhdr 18a, and memory 16b.

⟨*global* textbase 19d⟩≡ (97b)

```
uintptr textbase;
```

⟨initmemory() *Text segment initilisation* 19e⟩+≡ (19a) ◁ 19c 85f ▷

```
textbase = s->base;
```

Uses textbase 19d.

⟨initmemory() *Data segment initilisation* 19f⟩≡ (19a) 20b ▷

```
s = &memory.seg[Data];
s->type = Data;
s->base = t;
s->end = t+(d-t);
s->fileoff = fhdr.datoff;
s->fileend = s->fileoff + fhdr.datsz;
s->table = emalloc(((s->end - s->base)/BY2PG)*sizeof(byte*));
```

Uses BY2PG 18g, Data 15g, emalloc() 94a, fhdr 18a, and memory 16b.

⟨*global* `datasize` 20a⟩≡ (97b)

```
int datasize;
```

⟨`initmemory()` *Data segment initilisation* 20b⟩+≡ (19a) ◁19f

```
datasize = fhdr.datsz;
```

Uses `datasize` 20a and `fhdr` 18a.

⟨`initmemory()` *Bss segment initilisation* 20c⟩≡ (19a)

```
s = &memory.seg[Bss];
s->type = Bss;
s->base = d;
s->end = d+(b-d);
s->table = emalloc(((s->end - s->base)/BY2PG) * sizeof(byte*));
```

Uses `BY2PG` 18g, `Bss` 15g, `emalloc()` 94a, and `memory` 16b.

⟨`initmemory()` *Stack segment initilisation* 20d⟩≡ (19a)

```
s = &memory.seg[Stack];
s->type = Stack;
s->base = STACKTOP-STACKSIZE;
s->end = STACKTOP;
s->table = emalloc(((s->end - s->base)/BY2PG) * sizeof(byte*));
```

Uses `BY2PG` 18g, `STACKSIZE` 18g, `STACKTOP` 18g, `Stack` 15g, `emalloc()` 94a, and `memory` 16b.

4.3 initstk()

⟨*function* `initstk` 20e⟩≡ (101a)

```
void
initstk(int argc, char *argv[])
{
    ulong size;
    ulong sp, ap, tos;
    int i;
    char *p;

    tos = STACKTOP - sizeof(Tos)*2; /* we'll assume twice the host's is big enough */
    sp = tos;
    for (i = 0; i < sizeof(Tos)*2; i++)
        putmem_b(tos + i, 0);

    /*
     * pid is second word from end of tos and needs to be set for nsec().
     * we know arm is a 32-bit cpu, so we'll assume knowledge of the Tos
     * struct for now, and use our pid.
     */
    putmem_w(tos + 4*4 + 2*sizeof(ulong) + 3*sizeof(uvlong), getpid());

    /* Build exec stack */
    size = strlen(file)+1+BY2WD+BY2WD+BY2WD;
    for(i = 0; i < argc; i++)
        size += strlen(argv[i])+BY2WD+1;

    sp -= size;
    sp &= ~7;

    reg.r[0] = tos;
    reg.r[REGSP] = sp;
    reg.r[1] = STACKTOP-4; /* Plan 9 profiling clock (why & why in R1?) */
```

```
    /* Push argc */
    putmem_w(sp, argc+1);
    sp += BY2WD;

    /* Compute sizeof(argv) and push argv[0] */
    ap = sp+((argc+1)*BY2WD)+BY2WD;
    putmem_w(sp, ap);
    sp += BY2WD;

    /* Build argv[0] string into stack */
    for(p = file; *p; p++)
        putmem_b(ap++, *p);

    putmem_b(ap++, '\0');

    /* Loop through pushing the arguments */
    for(i = 0; i < argc; i++) {
        putmem_w(sp, ap);
        sp += BY2WD;
        for(p = argv[i]; *p; p++)
            putmem_b(ap++, *p);
        putmem_b(ap++, '\0');
    }
    /* Null terminate argv */
    putmem_w(sp, 0);

}
```

Uses BY2WD 18g, REGSP 15f, STACKTOP 18g, file 17a, putmem_b() 50b, putmem_w() 50d, and reg 15e.

4.4 cmd()

⟨*global* dot 21a⟩≡ (97b)

```
uintptr dot;
```

⟨cmd() *locals* 21b⟩≡ (21c) 67a ▷

```
char *p;
```

⟨*function* cmd 21c⟩≡ (102a)

```
void
cmd(void)
{
    ⟨cmd() locals 21b⟩

    dot = reg.r[REGPC];

    ⟨cmd() initialisation 90a⟩

    for(;;) {
        Bflush(bout);
        ⟨cmd() read and parse command and address from user input 67e⟩

        switch(*p) {
        ⟨cmd() command cases 22a⟩
        default:
            Bprint(bout, "?\n");
            break;
        }
    }
```

```
}
```

Uses REGPC 15f, bout 17d, dot 21a, and reg 15e.

⟨cmd() *command cases* 22a⟩≡ (21c) 68b ▷

```
case ':':
    colon(addr, p+1);
    break;
```

Uses colon() 22b.

⟨*function* colon 22b⟩≡ (102a)

```
void
colon(char *addr, char *cp)
{
    ⟨colon() locals 73a⟩

    cp = nextc(cp);

    switch(*cp) {
    ⟨colon() command which return cases 73c⟩
    /* These fall through to print the stopped address */
    ⟨colon() command cases 22c⟩
    default:
        Bprint(bout, "?\n");
        return;
    }

    dot = reg.r[REGPC];

    Bprint(bout, "%s at #%lux ", atbpt? "breakpoint": "stopped", dot);
    ⟨colon() print current instruction 73b⟩
    Bprint(bout, "\n");
}
```

Uses REGPC 15f, atbpt 82c, bout 17d, dot 21a, nextc() 68a, and reg 15e.

⟨colon() *command cases* 22c⟩≡ (22b) 73f ▷

```
case 'c':
    count = 0;
    atbpt = false;
    run();
    break;
```

Uses atbpt 82c, count 23a, false, and run() 23c.

Chapter 5

Instruction Interpreter

5.1 Instruction binary format

5.2 run()

⟨*global* count 23a⟩≡ (97b)

```
int count;
```

⟨Registers *other fields* 23b⟩≡ (15d) 28e ▷

```
uintptr  ar;    // reg.r[REGPC]

instruction instr;    // ifetch(reg.ar)
//enum<opcode>
int   instr_opcode; // arm_class(reg.instr)
Inst*  ip;    // &itab[reg.instr_opcode]
```

⟨*function* run 23c⟩≡ (99c)

```
void
run(void)
{
    bool execute;

    do {
        reg.ar = reg.r[REGPC];
        reg.instr = ifetch(reg.ar);

        reg.instr_opcode = arm_class(reg.instr);
        reg.ip = &itab[reg.instr_opcode];

        ⟨run() set reg.cond 42c⟩
        ⟨run() switch reg.compare_op to set execute 42f⟩

        if(execute) {
            ⟨run() profile current instruction class 85b⟩
            // !!the dispatch!!
            (*reg.ip->func)(reg.instr);

        }
        else
          if(trace) itrace("%s%s IGNORED",reg.ip->name, cond[reg.instr_cond]);

        reg.r[REGPC] += 4; // simple archi with fixed-length instruction :)

        ⟨run() check for breakpoints 84a⟩
```

```
    } while(--count);
}
```

Uses REGPC 15f, arm_class() 24a, cond<> 92d, count 23a, ifetch() 47d, itab 15b, itrace() 92b, reg 15e, and trace 92a.

5.3 arm_class()

⟨*function* arm_class 24a⟩≡ (99c)

```
int
arm_class(instruction w)
{
    // between 0 and 7
    int class;
    // enum<opcode>
    int op;
    ⟨arm_class() locals 24c⟩

    class = (w >> 25) & 0x7;
    switch(class) {
    ⟨arm_class() class cases 25a⟩
    default:
        op = OUNDEF;
        break;
    }
    return op;
}
```

Uses OUNDEF 14b.

5.4 Arithmetic and logic

5.4.1 Opcode extraction

⟨*arith/logic opcodes* 24b⟩≡ (14b) 25b ▷

```
OAND = 0,
OEOR = 1,
OSUB = 2,
ORSB = 3,
OADD = 4,
OADC = 5,
OSBC = 6,
ORSC = 7,
OTST = 8,
OTEQ = 9,
OCMP = 10,
OCMN = 11,
OORR = 12,
OMOV = 13,
OBIC = 14,
OMVN = 15,
```

⟨arm_class() *locals* 24c⟩≡ (24a)

```
int x;
```

⟨arm_class() *class cases* 25a⟩≡ (24a) 32a ▷

```
case 0: /* data processing r,r,r */
    x = ((w >> 4) & 0xf);
    ⟨arm_class() class 0, if x is 0x9 25c⟩
    ⟨arm_class() class 0, if x has 0x9 bits 35d⟩
    // else

    // the opcode! OAND/OADD/...
    op = (w >> 21) & 0xf;

    ⟨arm_class() class 0, adjust op for operands mode 25e⟩
    break;
```

⟨*arith/logic opcodes* 25b⟩+≡ (14b) ◁24b 25d ▷

```
OMUL    = 64,
OMULA   = 65,
```

⟨arm_class() *class 0, if x is 0x9* 25c⟩≡ (25a)

```
/* mul, swp, mull */
if(x == 0x9) {
    op = CMUL;
    ⟨arm_class() class 0, when x == 0x9, if bit 24 set 35f⟩
    ⟨arm_class() class 0, when x == 0x9, if bit 23 set 33b⟩
    if(w & (1<<21))
        op++;  /* mla */
    break;
}
```

Uses CMUL 25d.

⟨*arith/logic opcodes* 25d⟩+≡ (14b) ◁25b 25f ▷

```
CMUL    = 64,
```

⟨arm_class() *class 0, adjust op for operands mode* 25e⟩≡ (25a)

```
if(w & (1<<4))
  op += CARITH2;
else
 if((w & (31<<7)) || (w & (1<<5)))
  op += CARITH1;
// else op += CARITH0
```

Uses CARITH1 25f and CARITH2 25f.

⟨*arith/logic opcodes* 25f⟩+≡ (14b) ◁25d 32b ▷

```
CARITH0 = 0,  // r,r,r
CARITH1 = 16, // r<>#, r, r
CARITH2 = 32, // r<>r, r, r
```

5.4.2 Multiplication

⟨itab *elements* 25g⟩+≡ (15b) ◁15c 26c ▷

```
[OMUL]    =  { Imul, "MUL", Iarith },
[OMULA]   =  { Imula, "MULA", Iarith },
```

Uses Iarith 14c, Imul() 26a, Imula() 26b, OMUL 25b, and OMULA 25b.

⟨*function* Imul 26a⟩≡ (99c)

```
void
Imul(instruction inst)
{
    int rs, rd, rm;

    rd = (inst>>16) & 0xf;
    rs = (inst>>8) & 0xf;
    rm = inst & 0xf;

    if(rd == REGPC || rs == REGPC || rm == REGPC || rd == rm)
        undef(inst);

    reg.r[rd] = reg.r[rm]*reg.r[rs];

    ⟨Imul() trace ??⟩
}
```

Uses REGPC 15f, reg 15e, and undef() 93d.

⟨*function* Imula 26b⟩≡ (99c)

```
void
Imula(instruction inst)
{
    int rs, rd, rm, rn;

    rd = (inst>>16) & 0xf;
    rn = (inst>>12) & 0xf;
    rs = (inst>>8) & 0xf;
    rm = inst & 0xf;

    if(rd == REGPC || rn == REGPC || rs == REGPC || rm == REGPC || rd == rm)
        undef(inst);

    reg.r[rd] = reg.r[rm]*reg.r[rs] + reg.r[rn];

    ⟨Imula() trace ??⟩
}
```

Uses REGPC 15f, reg 15e, and undef() 93d.

5.4.3 Idp() and dpex()

⟨itab *elements* 26c⟩+≡ (15b) ◁25g 29b▷

```
//r,r,r
[OAND] =  { Idp0,  "AND", Iarith },
[OEOR] =  { Idp0,  "EOR", Iarith },
[OSUB] =  { Idp0,  "SUB", Iarith },
[ORSB] =  { Idp0,  "RSB", Iarith },
[OADD] =  { Idp0,  "ADD", Iarith },
[OADC] =  { Idp0,  "ADC", Iarith },
[OSBC] =  { Idp0,  "SBC", Iarith },
[ORSC] =  { Idp0,  "RSC", Iarith },
[OTST] =  { Idp0,  "TST", Iarith },
[OTEQ] =  { Idp0,  "TEQ", Iarith },
[OCMP] =  { Idp0,  "CMP", Iarith },
[OCMN] =  { Idp0,  "CMN", Iarith },
[OORR] =  { Idp0,  "ORR", Iarith },
[OMOV] =  { Idp0,  "MOV", Iarith },
[OBIC] =  { Idp0,  "BIC", Iarith },
```

```
[OMVN] =  { Idp0,  "MVN", Iarith },
```

Uses Iarith 14c, Idp0() 27a, OADC 24b, OADD 24b, OAND 24b, OBIC 24b, OCMN 24b, OCMP 24b, OEOR 24b, OMOV 24b, OMVN 24b, OORR 24b, ORSB 24b, ORSC 24b, OSBC 24b, OSUB 24b, OTEQ 24b, and OTST 24b.

⟨*function* Idp0 27a⟩≡ (99c)

```
/*
 * data processing instruction R,R,R
 */
void
Idp0(instruction inst)
{
    int rn, rd, rm;
    long o1, o2;

    rn = (inst>>16) & 0xf;
    rd = (inst>>12) & 0xf;
    rm = inst & 0xf;

    o1 = reg.r[rn];
    ⟨adjust o1 if rn is REGPC 27b⟩
    o2 = reg.r[rm];
    ⟨adjust o2 if rm is REGPC 27c⟩

    dpex(inst, o1, o2, rd);

    ⟨Idp0() trace ??⟩
    ⟨Idpx() compensate REGPC 27d⟩
}
```

Uses dpex() 27e and reg 15e.

⟨*adjust o1 if rn is REGPC* 27b⟩≡ (32d 30b 29c 27a)

```
if(rn == REGPC)
    o1 += 8;
```

Uses REGPC 15f.

⟨*adjust o2 if rm is REGPC* 27c⟩≡ (30b 29c 27a)

```
if(rm == REGPC)
    o2 += 8;
```

Uses REGPC 15f.

⟨Idpx() *compensate REGPC* 27d⟩≡ (32d 30b 29c 27a)

```
if(rd == REGPC)
    reg.r[rd] -= 4;
```

Uses REGPC 15f and reg 15e.

⟨*function* dpex 27e⟩≡ (99c)

```
void
dpex(instruction inst, long o1, long o2, int rd)
{
    bool cbit = false;

    switch((inst>>21) & 0xf) {
    ⟨dpex() switch arith/logic opcode cases 28a⟩
    }
    ⟨dpex() if Sbit 28d⟩
}
```

Uses false.

5.4.4 Boolean logic

⟨dpex() *switch arith/logic opcode cases* 28a⟩≡ (27e) 28b ▷

```
case  OAND:
    reg.r[rd] = o1 & o2;
    cbit = true;
    break;
case OORR:
    reg.r[rd] = o1 | o2;
    cbit = true;
    break;
case OEOR:
    reg.r[rd] = o1 ^ o2;
    cbit = true;
    break;
```

Uses OAND 24b, OEOR 24b, OORR 24b, reg 15e, and true.

⟨dpex() *switch arith/logic opcode cases* 28b⟩+≡ (27e) ◁ 28a 28f ▷

```
case OBIC:
    reg.r[rd] = o1 & ~o2;
    cbit = true;
    break;
```

Uses OBIC 24b, reg 15e, and true.

5.4.5 S bit

⟨*constant* Sbit 28c⟩≡ (18g)

```
Sbit  = 1<<20,
```

⟨dpex() *if Sbit* 28d⟩≡ (27e)

```
if(inst & Sbit) {
    if(cbit)
        reg.cbit = reg.cout;
    reg.cc1 = reg.r[rd];
    reg.cc2 = 0;
    reg.compare_op = CCcmp;
}
```

Uses CCcmp 42e, Sbit 28c, and reg 15e.

⟨Registers *other fields* 28e⟩+≡ (15d) ◁ 23b 42b ▷

```
int cbit; // carry bit?
int cout;
```

5.4.6 Addition and overflow

⟨dpex() *switch arith/logic opcode cases* 28f⟩+≡ (27e) ◁ 28b 34a ▷

```
case OADD:
    ⟨dpex() if calltree, when add operation 81c⟩
    reg.r[rd] = o1 + o2;

    if(inst & Sbit) {
        if((XCAST(o1) + XCAST(o2)) & (1LL << 32))
            reg.cbit = true;
        else
            reg.cbit = false;
        reg.cc1 = o2;
        reg.cc2 = -o1;
        reg.compare_op = CCcmp;
```

```
    }
    return;
```

Uses CCcmp 42e, OADD 24b, Sbit 28c, false, reg 15e, and true.

⟨*macro* XCAST 29a⟩≡ (99c)

```
#define XCAST(a) (uvlong)(ulong)a
```

5.4.7 shift()

⟨itab *elements* 29b⟩+≡ (15b) ◁26c 30a▷

```
// r<>#, r, r
[OAND +CARITH1] =  { Idp1,  "AND", Iarith },
[OEOR +CARITH1] =  { Idp1,  "EOR", Iarith },
[OSUB +CARITH1] =  { Idp1,  "SUB", Iarith },
[ORSB +CARITH1] =  { Idp1,  "RSB", Iarith },
[OADD +CARITH1] =  { Idp1,  "ADD", Iarith },
[OADC +CARITH1] =  { Idp1,  "ADC", Iarith },
[OSBC +CARITH1] =  { Idp1,  "SBC", Iarith },
[ORSC +CARITH1] =  { Idp1,  "RSC", Iarith },
[OTST +CARITH1] =  { Idp1,  "TST", Iarith },
[OTEQ +CARITH1] =  { Idp1,  "TEQ", Iarith },
[OCMP +CARITH1] =  { Idp1,  "CMP", Iarith },
[OCMN +CARITH1] =  { Idp1,  "CMN", Iarith },
[OORR +CARITH1] =  { Idp1,  "ORR", Iarith },
[OMOV +CARITH1] =  { Idp1,  "MOV", Iarith },
[OBIC +CARITH1] =  { Idp1,  "BIC", Iarith },
[OMVN +CARITH1] =  { Idp1,  "MVN", Iarith },
```

Uses CARITH1 25f, Iarith 14c, Idp1() 29c, OADC 24b, OADD 24b, OAND 24b, OBIC 24b, OCMN 24b, OCMP 24b, OEOR 24b, OMOV 24b, OMVN 24b, OORR 24b, ORSB 24b, ORSC 24b, OSBC 24b, OSUB 24b, OTEQ 24b, and OTST 24b.

⟨*function* Idp1 29c⟩≡ (99c)

```
/*
 * data processing instruction (R<>#),R,R
 */
void
Idp1(instruction inst)
{
    int rn, rd, rm, st, sc;
    long o1, o2;

    rn = (inst>>16) & 0xf;
    rd = (inst>>12) & 0xf;
    rm = inst & 0xf;
    st = (inst>>5) & 0x3;
    sc = (inst>>7) & 0x1f;

    o1 = reg.r[rn];
    ⟨adjust o1 if rn is REGPC 27b⟩
    o2 = reg.r[rm];
    ⟨adjust o2 if rm is REGPC 27c⟩

    o2 = shift(o2, st, sc, false);
    dpex(inst, o1, o2, rd);

    ⟨Idp1() trace ??⟩
    ⟨Idpx() compensate REGPC 27d⟩
}
```

Uses dpex() 27e, false, reg 15e, and shift() 30d.

⟨itab *elements* 30a⟩+≡ (15b) ◁29b 32c▷

```
// r<>r, r, r
[OAND +CARITH2] =  { Idp2,  "AND", Iarith },
[OEOR +CARITH2] =  { Idp2,  "EOR", Iarith },
[OSUB +CARITH2] =  { Idp2,  "SUB", Iarith },
[ORSB +CARITH2] =  { Idp2,  "RSB", Iarith },
[OADD +CARITH2] =  { Idp2,  "ADD", Iarith },
[OADC +CARITH2] =  { Idp2,  "ADC", Iarith },
[OSBC +CARITH2] =  { Idp2,  "SBC", Iarith },
[ORSC +CARITH2] =  { Idp2,  "RSC", Iarith },
[OTST +CARITH2] =  { Idp2,  "TST", Iarith },
[OTEQ +CARITH2] =  { Idp2,  "TEQ", Iarith },
[OCMP +CARITH2] =  { Idp2,  "CMP", Iarith },
[OCMN +CARITH2] =  { Idp2,  "CMN", Iarith },
[OORR +CARITH2] =  { Idp2,  "ORR", Iarith },
[OMOV +CARITH2] =  { Idp2,  "MOV", Iarith },
[OBIC +CARITH2] =  { Idp2,  "BIC", Iarith },
[OMVN +CARITH2] =  { Idp2,  "MVN", Iarith },
```

Uses CARITH2 25f, Iarith 14c, Idp2() 30b, OADC 24b, OADD 24b, OAND 24b, OBIC 24b, OCMN 24b, OCMP 24b, OEOR 24b, OMOV 24b, OMVN 24b, OORR 24b, ORSB 24b, ORSC 24b, OSBC 24b, OSUB 24b, OTEQ 24b, and OTST 24b.

⟨*function* Idp2 30b⟩≡ (99c)

```
/*
 * data processing instruction (R<>R),R,R
 */
void
Idp2(instruction inst)
{
    int rn, rd, rm, rs, st;
    long o1, o2, o3;

    rn = (inst>>16) & 0xf;
    rd = (inst>>12) & 0xf;
    rm = inst & 0xf;
    st = (inst>>5) & 0x3;
    rs = (inst>>8) & 0xf;

    o1 = reg.r[rn];
    ⟨adjust o1 if rn is REGPC 27b⟩
    o2 = reg.r[rm];
    ⟨adjust o2 if rm is REGPC 27c⟩
    o3 = reg.r[rs];
    ⟨adjust o3 if rs is REGPC 30c⟩

    o2 = shift(o2, st, o3, true);
    dpex(inst, o1, o2, rd);

    ⟨Idp2() trace ??⟩
    ⟨Idpx() compensate REGPC 27d⟩
}
```

Uses dpex() 27e, reg 15e, shift() 30d, and true.

⟨*adjust o3 if rs is REGPC* 30c⟩≡ (30b)

```
if(rs == REGPC)
    o3 += 8;
```

Uses REGPC 15f.

⟨*function* shift 30d⟩≡ (99c)

```
long
```

```
shift(long v, int st, int sc, bool isreg)
{
    ⟨shift() if sc is 0 31⟩
    else {
        switch(st) {
        case 0: /* logical left */
            reg.cout = (v >> (32 - sc)) & 1;
            v = v << sc;
            break;
        case 1: /* logical right */
            reg.cout = (v >> (sc - 1)) & 1;
            v = (ulong)v >> sc;
            break;
        case 2: /* arith right */
            if(sc >= 32) {
                reg.cout = (v >> 31) & 1;
                if(reg.cout)
                    v = 0xFFFFFFFF;
                else
                    v = 0;
            }
            else {
                reg.cout = (v >> (sc - 1)) & 1;
                v = (long)v >> sc;
            }
            break;
        case 3: /* rotate right */
            reg.cout = (v >> (sc - 1)) & 1;
            v = (v << (32-sc))
                 |
                ((ulong)v >> sc);
            break;
        }
    }
    return v;
}
```

Uses `reg` 15e.

⟨`shift()` *if sc is 0* 31⟩≡ (30d)

```
if(sc == 0) {
    switch(st) {
    case 0: /* logical left */
        reg.cout = reg.cbit;
        break;
    case 1: /* logical right */
        reg.cout = (v >> 31) & 1;
        break;
    case 2: /* arith right */
        reg.cout = reg.cbit;
        break;
    case 3: /* rotate right */
        if(isreg) {
            reg.cout = reg.cbit;
        }
        else {
            reg.cout = v & 1;
            v = ((ulong)v >> 1) | (reg.cbit << 31);
        }
    }
}
```

Uses reg 15e.

5.4.8 Immediate values

⟨arm_class() *class cases* 32a⟩+≡ (24a) ◁25a 34f▷

```
case 1: /* data processing i,r,r */
 op = CARITH3 + ((w >> 21) & 0xf);
 break;
```

Uses CARITH3 32b.

⟨*arith/logic opcodes* 32b⟩+≡ (14b) ◁25f 33a▷

```
CARITH3 = 48, // i,r,r
```

⟨itab *elements* 32c⟩+≡ (15b) ◁30a 33c▷

```
//i,r,r
[OAND +CARITH3] =  { Idp3,  "AND", Iarith },
[OEOR +CARITH3] =  { Idp3,  "EOR", Iarith },
[OSUB +CARITH3] =  { Idp3,  "SUB", Iarith },
[ORSB +CARITH3] =  { Idp3,  "RSB", Iarith },
[OADD +CARITH3] =  { Idp3,  "ADD", Iarith },
[OADC +CARITH3] =  { Idp3,  "ADC", Iarith },
[OSBC +CARITH3] =  { Idp3,  "SBC", Iarith },
[ORSC +CARITH3] =  { Idp3,  "RSC", Iarith },
[OTST +CARITH3] =  { Idp3,  "TST", Iarith },
[OTEQ +CARITH3] =  { Idp3,  "TEQ", Iarith },
[OCMP +CARITH3] =  { Idp3,  "CMP", Iarith },
[OCMN +CARITH3] =  { Idp3,  "CMN", Iarith },
[OORR +CARITH3] =  { Idp3,  "ORR", Iarith },
[OMOV +CARITH3] =  { Idp3,  "MOV", Iarith },
[OBIC +CARITH3] =  { Idp3,  "BIC", Iarith },
[OMVN +CARITH3] =  { Idp3,  "MVN", Iarith },
```

Uses CARITH3 32b, Iarith 14c, Idp3() 32d, OADC 24b, OADD 24b, OAND 24b, OBIC 24b, OCMN 24b, OCMP 24b, OEOR 24b, OMOV 24b, OMVN 24b, OORR 24b, ORSB 24b, ORSC 24b, OSBC 24b, OSUB 24b, OTEQ 24b, and OTST 24b.

⟨*function* Idp3 32d⟩≡ (99c)

```
/*
 * data processing instruction #<>#,R,R
 */
void
Idp3(instruction inst)
{
    int rn, rd, sc;
    long o1, o2;

    rn = (inst>>16) & 0xf;
    rd = (inst>>12) & 0xf;
    o1 = reg.r[rn];
    ⟨adjust o1 if rn is REGPC 27b⟩

    o2 = inst & 0xff;
    sc = (inst>>7) & 0x1e;
    o2 = (o2 >> sc) | (o2 << (32 - sc)); // rotate

    dpex(inst, o1, o2, rd);

    ⟨Idp3() trace ??⟩
    ⟨Idpx() compensate REGPC 27d⟩
}
```

Uses dpex() 27e and reg 15e.

5.4.9 64 bits target (signed/unsigned) multiplication

⟨*arith/logic opcodes* 33a⟩+≡ (14b) ◁32b

```
OMULLU  = 66,
OMULALU = 67,
OMULL   = 68,
OMULAL  = 69,
```

⟨`arm_class()` *class 0, when x == 0x9, if bit 23 set* 33b⟩≡ (25c)

```
if(w & (1<<23)) { /* mullu */
    op = CMUL+2;
    if(w & (1<<22)) /* mull */
        op = CMUL+4;
}
```

Uses `CMUL` 25d.

⟨`itab` *elements* 33c⟩+≡ (15b) ◁32c 35g▷

```
[OMULLU]  =  { Imull, "MULLU", Iarith },
[OMULALU] =  { Imull, "MULALU", Iarith },
[OMULL]   =  { Imull, "MULL",  Iarith },
[OMULAL]  =  { Imull, "MULAL", Iarith },
```

Uses `Iarith` 14c, `Imull()` 33d, `OMULAL` 33a, `OMULALU` 33a, `OMULL` 33a, and `OMULLU` 33a.

⟨*function* `Imull` 33d⟩≡ (99c)

```
void
Imull(instruction inst)
{
    vlong v;
    int rs, rd, rm, rn;

    rd = (inst>>16) & 0xf;
    rn = (inst>>12) & 0xf;
    rs = (inst>>8) & 0xf;
    rm = inst & 0xf;

    if(rd == REGPC || rn == REGPC || rs == REGPC || rm == REGPC
      || rd == rm || rn == rm || rd == rn
      )
        undef(inst);

    if(inst & (1<<22)){ // mull
        v = (vlong)reg.r[rm] * (vlong)reg.r[rs];
        if(inst & (1 << 21)) // mull and accumulate
            v += reg.r[rn];
    }else{ // mullu
        v = XCAST(reg.r[rm]) * XCAST(reg.r[rs]);
        if(inst & (1 << 21)) // mullu and accumulate
            v += (ulong)reg.r[rn];
    }
    reg.r[rd] = v >> 32;
    reg.r[rn] = v;

    ⟨Imull() trace ??⟩
}
```

Uses `REGPC` 15f, `reg` 15e, and `undef()` 93d.

5.4.10 Misc instructions

⟨dpex() *switch arith/logic opcode cases* 34a⟩+≡ (27e) ◁28f 34b▷

```
case OCMN:
    if(inst & Sbit) { // not always true?
        reg.cc1 = o1;
        reg.cc2 = -o2;
        reg.compare_op = CCcmp;
    }
    return;
```

Uses CCcmp 42e, OCMN 24b, Sbit 28c, and reg 15e.

⟨dpex() *switch arith/logic opcode cases* 34b⟩+≡ (27e) ◁34a 34c▷

```
case  ORSB:
    reg.r[rd] = o2 - o1;
    if(inst & Sbit) {
        reg.cc1 = o2;
        reg.cc2 = o1;
        reg.compare_op = CCcmp;
    }
    return;
```

Uses CCcmp 42e, ORSB 24b, Sbit 28c, and reg 15e.

⟨dpex() *switch arith/logic opcode cases* 34c⟩+≡ (27e) ◁34b 34d▷

```
case  OADC:
case  OSBC:
case  ORSC:
    undef(inst);
```

Uses OADC 24b, ORSC 24b, OSBC 24b, and undef() 93d.

⟨dpex() *switch arith/logic opcode cases* 34d⟩+≡ (27e) ◁34c 41e▷

```
case OMOV:
    reg.r[rd] = o2;
    cbit = true;
    break;
case OMVN:
    reg.r[rd] = ~o2;
    cbit = true;
    break;
```

Uses OMOV 24b, OMVN 24b, reg 15e, and true.

5.5 Memory

5.5.1 Opcode extraction

⟨*memory opcodes* 34e⟩≡ (14b) 35b▷

```
OLDW    = 70,
OLDB    = 71,
OSTW    = 72,
OSTB    = 73,
```

⟨arm_class() *class cases* 34f⟩+≡ (24a) ◁32a 35a▷

```
case 2: /* load/store byte/word i(r) */
 //                          OxxB?              OSTx?
 op = CMEM_BASIS + CMEM0 + ((w >> 22) & 0x1) + ((w >> 19) & 0x2);
 break;
```

Uses CMEM0 35b and CMEM_BASIS 35b.

⟨arm_class() *class cases* 35a⟩+≡ (24a) ◁34f 39b▷

```
case 3: /* load/store byte/word (r)(r) */
 //                              OxxB?              OSTx?
 op = CMEM_BASIS + CMEM1 + ((w >> 22) & 0x1) + ((w >> 19) & 0x2);
 break;
```

Uses CMEM1 35b and CMEM_BASIS 35b.

⟨*memory opcodes* 35b⟩+≡ (14b) ◁34e 35c▷

```
CMEM_BASIS = 70,
CMEM0 = 0, // i(r)
CMEM1 = 4, // (r),(r)
CMEM2 = 8, // byte signed or half word
```

⟨*memory opcodes* 35c⟩+≡ (14b) ◁35b 35e▷

```
OLDH    = 78,
OLDBU   = 79,
OSTH    = 80,
OSTBU   = 81,
```

⟨arm_class() *class 0, if x has 0x9 bits* 35d⟩≡ (25a)

```
if((x & 0x9) == 0x9) {  /* ld/st byte/half s/u */
     //                          OxxBU?              OSTx?
     op = CMEM_BASIS + CMEM2 + ((w >> 22) & 0x1) + ((w >> 19) & 0x2);
     break;
}
```

Uses CMEM2 35b and CMEM_BASIS 35b.

⟨*memory opcodes* 35e⟩+≡ (14b) ◁35c 39a▷

```
OSWPW = 82,
OSWPBU = 83,
```

⟨arm_class() *class 0, when x == 0x9, if bit 24 set* 35f⟩≡ (25c)

```
if(w & (1<<24)) {
    op = OSWPW;
    if(w & (1<<22))
         op = OSWPBU;
    break;
}
```

Uses OSWPBU 35e and OSWPW 35e.

5.5.2 Memory swap

⟨itab *elements* 35g⟩+≡ (15b) ◁33c 36a▷

```
[OSWPW] =   { Iswap,  "SWPW", Imem },
[OSWPBU] =  { Iswap,  "SWPBU",Imem },
```

Uses Imem 14c, Iswap() 35h, OSWPBU 35e, and OSWPW 35e.

⟨*function* Iswap 35h⟩≡ (99c)

```
void
Iswap(instruction inst)
{
    int rn, rd, rm;
    ulong address, value;
    bool bbit;

    bbit = inst & (1<<22); // BU?

    rn = (inst>>16) & 0xf;
```

```
    rd = (inst>>12) & 0xf;
    rm = (inst>>0) & 0xf;

    address = reg.r[rn];
    if(bbit) {
        value = getmem_b(address);
        putmem_b(address, reg.r[rm]);
    } else {
        value = getmem_w(address);
        putmem_w(address, reg.r[rm]);
    }
    reg.r[rd] = value;

    ⟨Iswap() trace ??⟩
}
```

Uses getmem_b() 48e, getmem_w() 49a, putmem_b() 50b, putmem_w() 50d, and reg 15e.

5.5.3 Load/store

⟨itab *elements* 36a⟩+≡ (15b) ◁35g 36b▷

```
// load/store w/ub i,r
[OLDW +CMEM0] =  { Imem1,  "MOVW", Imem },
[OLDB +CMEM0] =  { Imem1,  "MOVB", Imem },
[OSTW +CMEM0] =  { Imem1,  "MOVW", Imem },
[OSTB +CMEM0] =  { Imem1,  "MOVB", Imem },
```

Uses CMEM0 35b, Imem 14c, Imem1() 36c, OLDB 34e, OLDW 34e, OSTB 34e, and OSTW 34e.

⟨itab *elements* 36b⟩+≡ (15b) ◁36a 37a▷

```
// load/store r,r
[OLDW +CMEM1] =  { Imem1,  "MOVW", Imem },
[OLDB +CMEM1] =  { Imem1,  "MOVB", Imem },
[OSTW +CMEM1] =  { Imem1,  "MOVW", Imem },
[OSTB +CMEM1] =  { Imem1,  "MOVB", Imem },
```

Uses CMEM1 35b, Imem 14c, Imem1() 36c, OLDB 34e, OLDW 34e, OSTB 34e, and OSTW 34e.

⟨*function* Imem1 36c⟩≡ (99c)

```
/*
 * load/store word/byte
 */
void
Imem1(instruction inst)
{
    int rn, rd, off, rm, sc, st;
    ulong address, value;
    bool prebit, ubit, bbit, wbit, lbit, bit25;

    bit25 = inst & (1<<25); // rm or I?
    prebit = inst & (1<<24); // Pre indexing?
    ubit = inst & (1<<23); // Up offset?
    bbit = inst & (1<<22); // Byte or Word?
    wbit = inst & (1<<21); // Write back address in rn?
    lbit = inst & (1<<20); // LDR or STR?

    rn = (inst>>16) & 0xf;
    rd = (inst>>12) & 0xf;

    SET(st);
    SET(sc);
```

```
    SET(rm);

    if(bit25) {
        // rm<>I(...)
        rm = inst & 0xf;
        st = (inst>>5) & 0x3;
        sc = (inst>>7) & 0x1f;
        off = reg.r[rm];
        if(rm == REGPC)
            off += 8;
        off = shift(off, st, sc, false);
    } else {
        // I(...)
        off = inst & 0xfff;
    }

    if(!ubit)
        off = -off;
    if(rn == REGPC)
        off += 8;

    address = reg.r[rn];
    if(prebit)
        address += off;

    if(lbit) {
        // LDR
        if(bbit)
            value = getmem_b(address);
        else
            value = getmem_w(address);
        if(rd == REGPC)
            value -= 4;
        reg.r[rd] = value;
    } else {
        // STR
        value = reg.r[rd];
        if(rd == REGPC)
            value -= 4;
        if(bbit)
            putmem_b(address, value);
        else
            putmem_w(address, value);
    }
    if(!prebit || wbit)
        reg.r[rn] += off;

    ⟨Imem1() trace ??⟩
}
```

Uses REGPC 15f, false, getmem_b() 48e, getmem_w() 49a, putmem_b() 50b, putmem_w() 50d, reg 15e, and shift() 30d.

⟨itab *elements* 37a⟩+≡ (15b) ◁36b 39d▷

```
// load/store h/sb
[OLDH]  =  { Imem2,  "MOV", Imem },
[OLDBU] =  { Imem2,  "MOV", Imem },
[OSTH]  =  { Imem2,  "MOV", Imem },
[OSTBU] =  { Imem2,  "MOV", Imem },
```

Uses Imem 14c, Imem2() 37b, OLDBU 35c, OLDH 35c, OSTBU 35c, and OSTH 35c.

⟨*function* Imem2 37b⟩≡ (99c)

```
/*
 * load/store unsigned byte/half word
 */
void
Imem2(instruction inst)
{
    int rn, rd, off, rm;
    ulong address, value;
    bool prebit, ubit, hbit, sbit, wbit, lbit, bit22;

    prebit = inst & (1<<24); // Pre indexing?
    ubit = inst & (1<<23); // Up offset?
    bit22 = inst & (1<<22);
    wbit = inst & (1<<21); // Write back address in rn
    lbit = inst & (1<<20); // LDR or STR?

    sbit = inst & (1<<6); // Signed?
    hbit = inst & (1<<5); // Half word or byte?

    rn = (inst>>16) & 0xf;
    rd = (inst>>12) & 0xf;

    SET(rm);
    if(bit22) {
        // I(...)
        off = ((inst>>4) & 0xf0) | (inst & 0xf);
    } else {
        // rm(...)
        rm = inst & 0xf;
        off = reg.r[rm];
        if(rm == REGPC)
            off += 8;
    }
    if(!ubit)
        off = -off;
    if(rn == REGPC)
        off += 8;

    address = reg.r[rn];
    if(prebit)
        address += off;

    if(lbit) {
        // LDR
        if(hbit) {
            value = getmem_h(address);
            if(sbit && (value & 0x8000))
                value |= 0xffff0000;
        } else {
            value = getmem_b(address);
            if(value & 0x80)
                value |= 0xffffff00;
        }
        if(rd == REGPC)
            value -= 4;
        reg.r[rd] = value;
    } else {
        // STR
        value = reg.r[rd];
        if(rd == REGPC)
```

```
            value -= 4;
        if(hbit) {
            putmem_h(address, value);
        } else {
            putmem_b(address, value);
        }
    }
    if(!prebit || wbit)
        reg.r[rn] += off;

    ⟨Imem2() trace ??⟩
}
```

Uses REGPC 15f, getmem_b() 48e, getmem_h() 48f, putmem_b() 50b, putmem_h() 50c, and reg 15e.

5.5.4 Multi registers load/store

⟨*memory opcodes* 39a⟩+≡ (14b) ◁35e 39c▷

```
OLDM = 84,
OSTM = 85,
```

⟨arm_class() *class cases* 39b⟩+≡ (24a) ◁35a 40b▷

```
case 4: /* block data transfer (r)(r) */
 op = CBLOC + ((w >> 20) & 0x1);
 break;
```

Uses CBLOC 39c.

⟨*memory opcodes* 39c⟩+≡ (14b) ◁39a

```
CBLOC   = 84,
```

⟨itab *elements* 39d⟩+≡ (15b) ◁37a 41a▷

```
// block move r,r
[OLDM] =  { Ilsm,  "LDM", Imem },
[OSTM] =  { Ilsm,  "STM", Imem },
```

Uses Ilsm() 39e, Imem 14c, OLDM 39a, and OSTM 39a.

⟨*function* Ilsm 39e⟩≡ (99c)

```
void
Ilsm(instruction inst)
{
    bool prebit, ubit, sbit, wbit, lbit;
    int i, rn, reglist;
    ulong address, predelta, postdelta;

    prebit = (inst>>24) & 0x1;
    ubit = (inst>>23) & 0x1;
    sbit = (inst>>22) & 0x1;
    wbit = (inst>>21) & 0x1;
    lbit = (inst>>20) & 0x1;
    rn =   (inst>>16) & 0xf;
    reglist = inst & 0xffff;

    if(reglist & 0x8000)
        undef(reg.instr);
    if(sbit)
        undef(reg.instr);

    address = reg.r[rn];
```

```
    if(prebit) {
        predelta = 4;
        postdelta = 0;
    } else {
        predelta = 0;
        postdelta = 4;
    }
    if(ubit) {
        for (i = 0; i < 16; ++i) {
            if(!(reglist & (1 << i)))
                continue;
            address += predelta;
            if(lbit)
                reg.r[i] = getmem_w(address);
            else
                putmem_w(address, reg.r[i]);
            address += postdelta;
        }
    } else {
        for (i = 15; 0 <= i; --i) {
            if(!(reglist & (1 << i)))
                continue;
            address -= predelta;
            if(lbit)
                reg.r[i] = getmem_w(address);
            else
                putmem_w(address, reg.r[i]);
            address -= postdelta;
        }
    }
    if(wbit) {
        reg.r[rn] = address;
    }

    ⟨Ilsm() trace ??⟩
}
```

Uses `getmem_w()` 49a, `putmem_w()` 50d, `reg` 15e, and `undef()` 93d.

5.6 Control flow

5.6.1 Opcode extraction

⟨*branching opcodes* 40a⟩≡ (14b) 40c ▷

```
OB = 86,
OBL = 87,
```

⟨`arm_class()` *class cases* 40b⟩+≡ (24a) ◁ 39b 44b ▷

```
case 5: /* branch / branch link */
 op = CBRANCH + ((w >> 24) & 0x1);
 break;
```

Uses `CBRANCH` 40c.

⟨*branching opcodes* 40c⟩+≡ (14b) ◁ 40a

```
CBRANCH = 86,
```

5.6.2 Simple branching

⟨itab *elements* 41a⟩+≡ (15b) ◁39d 41c▷

```
// branch
[OB]  =  { Ib, "B", Ibranch },
```

Uses Ib() 41b, Ibranch 14c, and OB 40a.

⟨*function* Ib 41b⟩≡ (99c)

```
void
Ib(instruction inst)
{
    long v;

    v = inst & 0xffffff; // 24 bits
    v = reg.r[REGPC] + (v << 2) + 8;
    ⟨Ib() trace ??⟩
    reg.r[REGPC] = v - 4;
}
```

Uses REGPC 15f and reg 15e.

5.6.3 Branch and link

⟨itab *elements* 41c⟩+≡ (15b) ◁41a 44c▷

```
[OBL] =  { Ibl, "BL", Ibranch },
```

Uses Ibl() 41d, Ibranch 14c, and OBL 40a.

⟨*function* Ibl 41d⟩≡ (99c)

```
void
Ibl(instruction inst)
{
    long v;
    Symbol s;

    v = inst & 0xffffff;
    v = reg.r[REGPC] + (v << 2) + 8;
    ⟨Ibl() trace ??⟩
    ⟨Ibl() if calltree 81d⟩
    reg.r[REGLINK] = reg.r[REGPC] + 4;
    reg.r[REGPC] = v - 4;
}
```

Uses REGLINK 15f, REGPC 15f, and reg 15e.

5.6.4 Comparisons

⟨dpex() *switch arith/logic opcode cases* 41e⟩+≡ (27e) ◁34d 42a▷

```
case OSUB:
    reg.r[rd] = o1 - o2;
    // Fallthrough
case OCMP:
    if(inst & Sbit) {
        reg.cc1 = o1;
        reg.cc2 = o2;
        reg.compare_op = CCcmp;
    }
    return;
```

Uses CCcmp 42e, OCMP 24b, OSUB 24b, Sbit 28c, and reg 15e.

⟨dpex() *switch arith/logic opcode cases* 42a⟩+≡ (27e) ◁41e

```
case  OTST:
    if(inst & Sbit) {
        reg.cc1 = o1;
        reg.cc2 = o2;
        reg.compare_op = CCtst;
    }
    return;
case  OTEQ:
    if(inst & Sbit) { // not always true?
        reg.cc1 = o1;
        reg.cc2 = o2;
        reg.compare_op = CCteq;
    }
    return;
```

Uses CCteq 42e, CCtst 42e, OTEQ 24b, OTST 24b, Sbit 28c, and reg 15e.

5.6.5 ARM Conditional execution

⟨Registers *other fields* 42b⟩+≡ (15d) ◁28e 42d▷

```
// actually only 4 bits (16 possibilities)
int instr_cond;
```

⟨run() *set reg.cond* 42c⟩≡ (23c)

```
reg.instr_cond = (reg.instr>>28) & 0xf;
```

Uses reg 15e.

⟨Registers *other fields* 42d⟩+≡ (15d) ◁42b 42g▷

```
// enum<compare_op>
int compare_op;
```

⟨*enum* compare_op 42e⟩≡ (95)

```
enum compare_op
{
    CCcmp,
    CCtst,
    CCteq,
};
```

⟨run() *switch* reg.compare_op *to set* execute 42f⟩≡ (23c)

```
switch(reg.compare_op) {
case CCcmp:
    execute = runcmp(); // use reg.instr_cond
    break;
case CCteq:
    execute = runteq();
    break;
case CCtst:
    execute = runtst();
    break;
default:
    Bprint(bout, "unimplemented compare operation %x\n",
        reg.compare_op);
    return;
}
```

Uses CCcmp 42e, CCteq 42e, CCtst 42e, bout 17d, reg 15e, runcmp() 43a, runteq() 43c, and runtst() 43d.

⟨Registers *other fields* 42g⟩+≡ (15d) ◁42d

```
long cc1;
long cc2;
```

⟨*function* runcmp 43a⟩≡ (99c)

```
bool
runcmp(void)
{
    switch(reg.instr_cond) {
    case 0x0: /* eq */ return (reg.cc1 == reg.cc2);
    case 0x1: /* ne */ return (reg.cc1 != reg.cc2);
    case 0x2: /* hs */ return ((ulong)reg.cc1 >= (ulong)reg.cc2);
    case 0x3: /* lo */ return ((ulong)reg.cc1 < (ulong)reg.cc2);
    case 0x4: /* mi */ return (reg.cc1 - reg.cc2 < 0);
    case 0x5: /* pl */ return (reg.cc1 - reg.cc2 >= 0);
    case 0x8: /* hi */ return ((ulong)reg.cc1 > (ulong)reg.cc2);
    case 0x9: /* ls */ return ((ulong)reg.cc1 <= (ulong)reg.cc2);
    case 0xa: /* ge */ return (reg.cc1 >= reg.cc2);
    case 0xb: /* lt */ return (reg.cc1 < reg.cc2);
    case 0xc: /* gt */ return (reg.cc1 > reg.cc2);
    case 0xd: /* le */ return (reg.cc1 <= reg.cc2);

    case 0xe: /* al */ return true;
    case 0xf: /* nv */ return false;
    default:
        Bprint(bout, "unimplemented condition prefix %x (%ld %ld)\n",
            reg.instr_cond, reg.cc1, reg.cc2);
        undef(reg.instr);
        return false;
    }
}
```

Uses bout 17d, false, reg 15e, true, and undef() 93d.

⟨*constant* SIGNBIT 43b⟩≡ (18g)

```
SIGNBIT  = 0x80000000,
```

⟨*function* runteq 43c⟩≡ (99c)

```
bool
runteq(void)
{
    long res = reg.cc1 ^ reg.cc2;

    switch(reg.instr_cond) {
    case 0x0: /* eq */ return res == 0;
    case 0x1: /* ne */ return res != 0;
    case 0x4: /* mi */ return (res & SIGNBIT) != 0;
    case 0x5: /* pl */ return (res & SIGNBIT) == 0;
    case 0xe: /* al */ return true;
    case 0xf: /* nv */ return false;
    default:
        Bprint(bout, "unimplemented condition prefix %x (%ld %ld)\n",
            reg.instr_cond, reg.cc1, reg.cc2);
        undef(reg.instr);
        return false;
    }
}
```

Uses SIGNBIT 43b, bout 17d, false, reg 15e, true, and undef() 93d.

⟨*function* runtst 43d⟩≡ (99c)

```
bool
runtst(void)
{
    long res = reg.cc1 & reg.cc2;
```

```
    switch(reg.instr_cond) {
    case 0x0: /* eq */ return res == 0;
    case 0x1: /* ne */ return res != 0;
    case 0x4: /* mi */ return (res & SIGNBIT) != 0;
    case 0x5: /* pl */ return (res & SIGNBIT) == 0;
    case 0xe: /* al */ return true;
    case 0xf: /* nv */ return false;
    default:
        Bprint(bout, "unimplemented condition prefix %x (%ld %ld)\n",
            reg.instr_cond, reg.cc1, reg.cc2);
        undef(reg.instr);
        return false;
    }
}
```

Uses `SIGNBIT` 43b, `bout` 17d, `false`, `reg` 15e, `true`, and `undef()` 93d.

5.7 Software interrupt

⟨*syscall opcodes* 44a⟩≡ (14b)

```
OSWI = 88,
```

⟨`arm_class()` *class cases* 44b⟩+≡ (24a) ◁40b

```
case 7: /* coprocessor crap */ // and syscall
 if((w >> 25) & 0x1)
   op = OSWI;
 else
   op = OUNDEF; // coprocessor stuff not handled
 break;
```

Uses `OSWI` 44a and `OUNDEF` 14b.

⟨`itab` *elements* 44c⟩+≡ (15b) ◁41c

```
[OSWI] =  { Ssyscall,  "SWI", Isyscall },
```

Uses `Isyscall` 14c, `OSWI` 44a, and `Ssyscall()` 53a.

5.8 Unimplemented instructions

Chapter 6

Memory

6.1 page_of_vaddr()

⟨*function* page_of_vaddr 45⟩≡ (99a)

```
void*
page_of_vaddr(uintptr addr)
{
    Segment *s, *es;
    int off, foff, l, n;
    byte **p, *a;

    ⟨page_of_vaddr() TLB handling 47b⟩

    es = &memory.seg[Nseg];
    for(s = memory.seg; s < es; s++) {
        if(addr >= s->base && addr < s->end) {
            s->refs++;
            off = (addr - s->base)/BY2PG;
            p = &s->table[off];

            if(*p)
                return *p;

            // else page fault! no allocated memory there yet
            s->rss++;

            switch(s->type) {
            ⟨page_of_vaddr() page fault, switch segment type cases 46a⟩
            default:
                fatal(false, "page_of_vaddr");
            }
        }
    }
    // reach here if didn't find any segment with relevant range
    Bprint(bout, "User TLB miss vaddr 0x%.8lux\n", addr);
    Bflush(bout);
    longjmp(errjmp, 0);
    return nil;  /*to stop compiler whining*/
}
```

Uses BY2PG 18g, Nseg 15g, bout 17d, errjmp 93b, false, fatal() 93a, memory 16b, and nil.

6.2 Page faults

⟨page_of_vaddr() *page fault, switch segment type cases* 46a⟩≡ (45) 46b ▷

```
case Text:
    *p = emalloc(BY2PG);
    if(seek(text, s->fileoff+(off*BY2PG), 0) < 0)
        fatal(true, "page_of_vaddr text seek");
    if(read(text, *p, BY2PG) < 0)
        fatal(true, "page_of_vaddr text read");
    return *p;
```

Uses BY2PG 18g, Text 15g, emalloc() 94a, fatal() 93a, seek(), text 17b, and true.

⟨page_of_vaddr() *page fault, switch segment type cases* 46b⟩+≡ (45) ◁ 46a 46c ▷

```
case Data:
    *p = emalloc(BY2PG);
    foff = s->fileoff+(off*BY2PG);
    if(seek(text, foff, 0) < 0)
        fatal(true, "page_of_vaddr text seek");
    n = read(text, *p, BY2PG);
    if(n < 0)
        fatal(true, "page_of_vaddr text read");
    if(foff + n > s->fileend) {
        l = BY2PG - (s->fileend-foff);
        a = *p+(s->fileend-foff);
        memset(a, 0, l);
    }
    return *p;
```

Uses BY2PG 18g, Data 15g, emalloc() 94a, fatal() 93a, seek(), text 17b, and true.

⟨page_of_vaddr() *page fault, switch segment type cases* 46c⟩+≡ (45) ◁ 46b

```
case Bss:
case Stack:
    *p = emalloc(BY2PG);
    return *p;
```

Uses BY2PG 18g, Bss 15g, Stack 15g, and emalloc() 94a.

6.3 Tlb

⟨*struct* Tlb 46d⟩≡ (95)

```
struct Tlb
{
    bool on;   /* Being updated */
    int  tlbsize;  /* Number of entries */
    // all pointers in array are at page granularity
    uintptr tlbent[Nmaxtlb]; /* Virtual address tags */

    int hit;   /* Number of successful tag matches */
    int miss;   /* Number of failed tag matches */
};
```

Uses Nmaxtlb 46e.

⟨*constant* Nmaxtlb 46e⟩≡ (95)

```
#define Nmaxtlb 64
```

⟨*global* tlb 46f⟩≡ (97b)

```
Tlb tlb;
```

⟨main() *tlb initialisation* 47a⟩≡ (17e)

```
tlb.on = true;
tlb.tlbsize = 24;
```

Uses tlb 46f and true.

⟨page_of_vaddr() *TLB handling* 47b⟩≡ (45)

```
if(tlb.on)
    dotlb(addr);
```

Uses dotlb() 47c and tlb 46f.

⟨*function* dotlb 47c⟩≡ (99a)

```
void
dotlb(uintptr vaddr)
{
    ulong *l, *e;

    vaddr &= ~(BY2PG-1);

    e = &tlb.tlbent[tlb.tlbsize];
    for(l = tlb.tlbent; l < e; l++)
        if(*l == vaddr) {
            tlb.hit++;
            return;
        }

    tlb.miss++;
    tlb.tlbent[lnrand(tlb.tlbsize)] = vaddr;
}
```

Uses BY2PG 18g and tlb 46f.

6.4 ifetch()

⟨*function* ifetch 47d⟩≡ (99a)

```
instruction
ifetch(uintptr addr)
{
    byte *va;

    if(addr&3) {
        Bprint(bout, "Address error (I-fetch) vaddr %.8lux\n", addr);
        longjmp(errjmp, 0);
    }

    ⟨ifetch() instruction cache handling 48c⟩
    iprof[(addr-textbase)/PROFGRAN]++;

    va = page_of_vaddr(addr); // get page
    va += addr&(BY2PG-1); // restore offset in page

    return va[3]<<24 | va[2]<<16 | va[1]<<8 | va[0];
}
```

Uses BY2PG 18g, PROFGRAN 85e, bout 17d, errjmp 93b, iprof 85d, page_of_vaddr() 45, and textbase 19d.

6.5 The instruction cache

⟨*struct* Icache 48a⟩≡ (95)

```
struct Icache
{
    bool on;   /* Turned on */

    int  linesize;  /* Line size in bytes */
    int  stall;   /* Cache stalls */
    int* lines;   /* Tag array */
    int* (*hash)(ulong); /* Hash function */
    char* hashtext;  /* What the function looks like */
};
```

⟨*global* icache 48b⟩≡ (97b)

```
Icache icache;
```

⟨ifetch() *instruction cache handling* 48c⟩≡ (47d)

```
if(icache.on)
    updateicache(addr);
```

Uses icache 48b and updateicache() 48d.

⟨*function* updateicache 48d⟩≡ (97a)

```
void
updateicache(uintptr addr)
{
    USED(addr);
}
```

6.6 getmem_xxx()

⟨*function* getmem_b 48e⟩≡ (99a)

```
byte
getmem_b(uintptr addr)
{
    byte *va;

    ⟨getmem_x() if membpt 84c⟩

    va = page_of_vaddr(addr);
    va += addr&(BY2PG-1);
    return va[0];
}
```

Uses BY2PG 18g and page_of_vaddr() 45.

⟨*function* getmem_h 48f⟩≡ (99a)

```
ushort
getmem_h(uintptr addr)
{
    byte *va;
    ulong w;

    if(addr&1) {
        w = getmem_h(addr & ~1);
        while(addr & 1) {
            w = (w>>8) | (w<<8);
            addr--;
        }
```

```
        return w;
    }
    ⟨getmem_x() if membpt 84c⟩

    va = page_of_vaddr(addr);
    va += addr&(BY2PG-1);

    return va[1]<<8 | va[0];
}
```

Uses BY2PG 18g, getmem_h() 48f, and page_of_vaddr() 45.

⟨*function* getmem_w 49a⟩≡ (99a)

```
ulong
getmem_w(uintptr addr)
{
    byte *va;
    ulong w;

    if(addr&3) {
        w = getmem_w(addr & ~3);
        while(addr & 3) {
            w = (w>>8) | (w<<24);
            addr--;
        }
        return w;
    }
    ⟨getmem_x() if membpt 84c⟩

    va = page_of_vaddr(addr);
    va += addr&(BY2PG-1);

    return va[3]<<24 | va[2]<<16 | va[1]<<8 | va[0];
}
```

Uses BY2PG 18g, getmem_w() 49a, and page_of_vaddr() 45.

⟨*function* getmem_v 49b⟩≡ (99a)

```
uvlong
getmem_v(uintptr addr)
{
    return ((uvlong)getmem_w(addr+4) << 32) | getmem_w(addr);
}
```

Uses getmem_w() 49a.

⟨*function* getmem_2 49c⟩≡ (99a)

```
ulong
getmem_2(uintptr addr)
{
    ulong val;
    int i;

    val = 0;
    for(i = 0; i < 2; i++)
        val = (val>>8) | (getmem_b(addr++)<<16);
    return val;
}
```

Uses getmem_b() 48e.

⟨*function* getmem_4 50a⟩≡ (99a)

```
ulong
getmem_4(uintptr addr)
{
    ulong val;
    int i;

    val = 0;
    for(i = 0; i < 4; i++)
        val = (val>>8) | (getmem_b(addr++)<<24);
    return val;
}
```

Uses getmem_b() 48e.

6.7 putmem_xxx()

⟨*function* putmem_b 50b⟩≡ (99a)

```
void
putmem_b(uintptr addr, byte data)
{
    byte *va;

    va = page_of_vaddr(addr);
    va += addr&(BY2PG-1);
    va[0] = data;

    ⟨putmem_x() if membpt 84d⟩
}
```

Uses BY2PG 18g and page_of_vaddr() 45.

⟨*function* putmem_h 50c⟩≡ (99a)

```
void
putmem_h(uintptr addr, ushort data)
{
    byte *va;

    if(addr&1) {
        Bprint(bout, "Address error (Store) vaddr %.8lux\n", addr);
        longjmp(errjmp, 0);
    }

    va = page_of_vaddr(addr);
    va += addr&(BY2PG-1);

    va[1] = data>>8;
    va[0] = data;

    ⟨putmem_x() if membpt 84d⟩
}
```

Uses BY2PG 18g, bout 17d, errjmp 93b, and page_of_vaddr() 45.

⟨*function* putmem_w 50d⟩≡ (99a)

```
void
putmem_w(uintptr addr, ulong data)
{
    byte *va;
```

```
    if(addr&3) {
        Bprint(bout, "Address error (Store) vaddr %.8lux\n", addr);
        longjmp(errjmp, 0);
    }

    va = page_of_vaddr(addr);
    va += addr&(BY2PG-1);

    va[3] = data>>24;
    va[2] = data>>16;
    va[1] = data>>8;
    va[0] = data;

    ⟨putmem_x() if membpt 84d⟩
}
```

Uses BY2PG 18g, bout 17d, errjmp 93b, and page_of_vaddr() 45.

⟨*function* putmem_v 51⟩≡ (99a)

```
void
putmem_v(uintptr addr, uvlong data)
{
    putmem_w(addr, data); /* two stages, to catch brkchk */
    putmem_w(addr+4, data>>32);
}
```

Uses putmem_w() 50d.

Chapter 7

Syscalls Emulation

⟨*global* systab 52⟩≡ (102b)

```
void (*systab[])(void) =
{
    [NOP]  sysnop,

    [RFORK]  sysrfork,
    [EXEC]  sysexec,
    [EXITS]  sysexits,
    [AWAIT]  sysawait,

    [BRK]  sysbrk,

    [OPEN]  sysopen,
    [CLOSE]  sysclose,
    [PREAD]  syspread,
    [PWRITE] syspwrite,
    [SEEK]  sysseek,

    [CREATE] syscreate,
    [REMOVE] sysremove,
    [CHDIR]  syschdir,
    [FD2PATH] sysfd2path,
    [STAT]  sysstat,
    [FSTAT]  sysfstat,
    [WSTAT]  syswstat,
    [FWSTAT] sysfwstat,

    [BIND]  sysbind,
    [MOUNT]  sysmount,
    [UNMOUNT] sysunmount,

    [SLEEP]  syssleep,
    [ALARM]  sysalarm,

    [PIPE]  syspipe,
    [NOTIFY] sysnotify,
    [NOTED]  sysnoted,

    [SEGATTACH] syssegattach,
    [SEGDETACH] syssegdetach,
    [SEGFREE] syssegfree,
    [SEGFLUSH] syssegflush,
    [SEGBRK] syssegbrk,
```

```
    [RENDEZVOUS] sysrendezvous,

    [DUP]  sysdup,
    [FVERSION] sysfversion,
    [FAUTH]  sysfauth,

    [ERRSTR] syserrstr,
};
```

Uses ALARM, AWAIT, BIND, BRK, CHDIR, CLOSE, CREATE, DUP, ERRSTR, EXEC, EXITS, FAUTH, FD2PATH, FSTAT, FVERSION, FWSTAT, MOUNT, NOP, NOTED, NOTIFY, OPEN, PIPE, PREAD, PWRITE, REMOVE, RENDEZVOUS, RFORK, SEEK, SEGATTACH, SEGBRK, SEGDETACH, SEGFLUSH, SEGFREE, SLEEP, STAT, UNMOUNT, WSTAT, sysalarm() 65f, sysawait() 63e, sysbind() 61a, sysbrk() 55a, syschdir() 60c, sysclose() 56a, syscreate() 59b, sysdup() 61b, syserrstr() 54c, sysexec() 63d, sysexits() 62b, sysfauth() 66a, sysfd2path() 60b, sysfstat() 59a, sysfversion() 66b, sysfwstat() 64b, sysmount() 65d, sysnop() 54a, sysnoted() 64c, sysnotify() 63b, sysopen() 55b, syspipe() 62d, syspread() 57a, syspwrite() 58b, sysremove() 60a, sysrendezvous() 65c, sysrfork() 63c, sysseek() 57b, syssegattach() 64d, syssegbrk() 65b, syssegdetach() 64e, syssegflush() 65a, syssegfree() 64f, syssleep() 62c, sysstat() 58c, systab 52, sysunmount() 65e, and syswstat() 64a.

⟨*function* Ssyscall 53a⟩≡ (102b)

```
void
Ssyscall(instruction _unused)
{
    int call;
    USED(_unused);

    call = reg.r[REGARG];

    if(call < 0 || call >= nelem(systab) || systab[call] == nil) {
        Bprint(bout, "bad system call %d (%#ux)\n", call, call);
        dumpreg();
        Bflush(bout);
        return;
    }

    if(trace)
        itrace("SWI\t%s", sysctab[call]);

    // dispatch!
    (*systab[call])();

    Bflush(bout);
}
```

Uses REGARG 15f, bout 17d, dumpreg() 78a, itrace() 92b, nelem, nil, reg 15e, sysctab, systab 52, and trace 92a.

7.1 memio()

⟨*enum* memxxx 53b⟩≡ (95)

```
// for memio()
enum
{
    MemRead,
    MemReadstring,
    MemWrite,
};
```

⟨*function* memio 53c⟩≡ (99a)

```
char *
memio(char *mb, uintptr mem, int size, int dir)
{
```

```
    int i;
    char *buf, c;

    if(mb == nil)
        mb = emalloc(size);

    buf = mb;
    switch(dir) {
    case MemRead:
        while(size--)
            *mb++ = getmem_b(mem++);
        break;
    case MemReadstring:
        for(;;) {
            if(size-- == 0) {
                Bprint(bout, "memio: user/kernel copy too long for arm\n");
                longjmp(errjmp, 0);
            }
            c = getmem_b(mem++);
            *mb++ = c;
            if(c == '\0')
                break;
        }
        break;
    case MemWrite:
        for(i = 0; i < size; i++)
            putmem_b(mem++, *mb++);
        break;
    default:
        fatal(false, "memio");
    }
    return buf;
}
```

Uses MemRead 53b, MemReadstring 53b, MemWrite 53b, bout 17d, emalloc() 94a, errjmp 93b, false, fatal() 93a, getmem_b() 48e, nil, and putmem_b() 50b.

7.2 Nop and errstr syscalls

⟨*function* sysnop 54a⟩≡ (102b)

```
void
sysnop(void)
{
    Bprint(bout, "nop system call %s\n", sysctab[reg.r[1]]);
    ⟨sysnop strace 79b⟩
}
```

Uses bout 17d, reg 15e, and sysctab.

⟨*global* errbuf 54b⟩≡ (102b)

```
char  errbuf[ERRMAX];
```

Uses ERRMAX.

⟨*function* syserrstr 54c⟩≡ (102b)

```
void
syserrstr(void)
{
    ulong str;
    int n;
```

```
    str = getmem_w(reg.r[REGSP]+4);
    n = getmem_w(reg.r[REGSP]+8);
    if(sysdbg)
        itrace("errstr(0x%lux, 0x%lux)", str, n);

    if(n > strlen(errbuf)+1)
        n = strlen(errbuf)+1;
    memio(errbuf, str, n, MemWrite);
    strcpy(errbuf, "no error");
    reg.r[REGRET] = n;

}
```

Uses MemWrite 53b, REGRET 15f, REGSP 15f, errbuf 54b, getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, and sysdbg 79a.

7.3 Memory syscalls

⟨*function* sysbrk 55a⟩≡ (102b)

```
void
sysbrk(void)
{
    ulong addr, osize, nsize;
    Segment *s;

    addr = getmem_w(reg.r[REGSP]+4);
    if(sysdbg)
        itrace("brk(0x%lux)", addr);

    reg.r[REGRET] = -1;
    if(addr < memory.seg[Data].base+datasize) {
        strcpy(errbuf, "address below segment");
        return;
    }
    if(addr > memory.seg[Stack].base) {
        strcpy(errbuf, "segment too big");
        return;
    }
    s = &memory.seg[Bss];
    if(addr > s->end) {
        osize = ((s->end-s->base)/BY2PG)*sizeof(byte*);
        addr = ((addr)+(BY2PG-1))&~(BY2PG-1);
        s->end = addr;
        nsize = ((s->end-s->base)/BY2PG)*sizeof(byte*);
        s->table = erealloc(s->table, osize, nsize);
    }

    reg.r[REGRET] = 0;
}
```

Uses BY2PG 18g, Bss 15g, Data 15g, REGRET 15f, REGSP 15f, Stack 15g, datasize 20a, erealloc() 94b, errbuf 54b, getmem_w() 49a, itrace() 92b, memory 16b, reg 15e, and sysdbg 79a.

7.4 File and IO syscalls

⟨*function* sysopen 55b⟩≡ (102b)

```
void
sysopen(void)
```

```
{
    char file[1024];
    int n;
    ulong mode, name;

    name = getmem_w(reg.r[REGSP]+4);
    mode = getmem_w(reg.r[REGSP]+8);
    memio(file, name, sizeof(file), MemReadstring);

    n = open(file, mode);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);

    if(sysdbg)
        itrace("open(0x%lux='%s', 0x%lux) = %d", name, file, mode, n);

    reg.r[REGRET] = n;
};
```

Uses MemReadstring 53b, REGRET 15f, REGSP 15f, errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, open(), reg 15e, and sysdbg 79a.

⟨*function* sysclose 56a⟩≡ (102b)

```
void
sysclose(void)
{
    int n;
    ulong fd;

    fd = getmem_w(reg.r[REGSP]+4);
    if(sysdbg)
        itrace("close(%d)", fd);

    n = close(fd);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);
    reg.r[REGRET] = n;
}
```

Uses REGRET 15f, REGSP 15f, close(), errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, reg 15e, and sysdbg 79a.

⟨*function* sysread 56b⟩≡ (102b)

```
void
sysread(vlong offset)
{
    int fd;
    ulong size, a;
    char *buf, *p;
    int n, cnt, c;

    fd = getmem_w(reg.r[REGSP]+4);
    a = getmem_w(reg.r[REGSP]+8);
    size = getmem_w(reg.r[REGSP]+12);

    buf = emalloc(size);
    if(fd == 0) {
        print("\nstdin>>");
        p = buf;
        n = 0;
        cnt = size;
        while(cnt) {
            c = Bgetc(bin);
```

```
            if(c <= 0)
                break;
            *p++ = c;
            n++;
            cnt--;
            if(c == '\n')
                break;
        }
    }
    else
        n = pread(fd, buf, size, offset);

    if(n < 0)
        errstr(errbuf, sizeof errbuf);
    else
        memio(buf, a, n, MemWrite);

    if(sysdbg)
        itrace("read(%d, 0x%lux, %d, 0x%llx) = %d", fd, a, size, offset, n);

    free(buf);
    reg.r[REGRET] = n;
}
```

Uses MemWrite 53b, REGRET 15f, REGSP 15f, bin 17d, emalloc() 94a, errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, pread(), reg 15e, and sysdbg 79a.

⟨*function* syspread 57a⟩≡ (102b)

```
void
syspread(void)
{
    sysread(getmem_v(reg.r[REGSP]+16));
}
```

Uses REGSP 15f, getmem_v() 49b, reg 15e, and sysread() 56b.

⟨*function* sysseek 57b⟩≡ (102b)

```
void
sysseek(void)
{
    int fd;
    ulong mode;
    ulong retp;
    vlong v;

    retp = getmem_w(reg.r[REGSP]+4);
    fd = getmem_w(reg.r[REGSP]+8);
    v = getmem_v(reg.r[REGSP]+16);
    mode = getmem_w(reg.r[REGSP]+20);
    if(sysdbg)
        itrace("seek(%d, %lld, %d)", fd, v, mode);

    v = seek(fd, v, mode);
    if(v < 0)
        errstr(errbuf, sizeof errbuf);

    putmem_v(retp, v);
}
```

Uses REGSP 15f, errbuf 54b, errstr(), getmem_v() 49b, getmem_w() 49a, itrace() 92b, putmem_v() 51, reg 15e, seek(), and sysdbg 79a.

⟨*function* syswrite 58a⟩≡ (102b)

```
void
syswrite(vlong offset)
{
    int fd;
    ulong size, a;
    char *buf;
    int n;

    fd = getmem_w(reg.r[REGSP]+4);
    a = getmem_w(reg.r[REGSP]+8);
    size = getmem_w(reg.r[REGSP]+12);

    Bflush(bout);
    buf = memio(0, a, size, MemRead);
    n = pwrite(fd, buf, size, offset);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);

    if(sysdbg)
        itrace("write(%d, %lux, %d, 0x%llx) = %d", fd, a, size, offset, n);

    free(buf);

    reg.r[REGRET] = n;
}
```

Uses MemRead 53b, REGRET 15f, REGSP 15f, bout 17d, errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, pwrite(), reg 15e, and sysdbg 79a.

⟨*function* syspwrite 58b⟩≡ (102b)

```
void
syspwrite(void)
{
    syswrite(getmem_v(reg.r[REGSP]+16));
}
```

Uses REGSP 15f, getmem_v() 49b, reg 15e, and syswrite() 58a.

⟨*function* sysstat 58c⟩≡ (102b)

```
void
sysstat(void)
{
    char nambuf[1024];
    byte buf[STATMAX];
    ulong edir, name;
    int n;

    name = getmem_w(reg.r[REGSP]+4);
    edir = getmem_w(reg.r[REGSP]+8);
    n = getmem_w(reg.r[REGSP]+12);
    memio(nambuf, name, sizeof(nambuf), MemReadstring);
    if(sysdbg)
        itrace("stat(0x%lux='%s', 0x%lux, 0x%lux)", name, nambuf, edir, n);
    if(n > sizeof buf)
        errstr(errbuf, sizeof errbuf);
    else{
        n = stat(nambuf, buf, n);
        if(n < 0)
            errstr(errbuf, sizeof errbuf);
        else
            memio((char*)buf, edir, n, MemWrite);
```

```
    }
    reg.r[REGRET] = n;
}
```

Uses MemReadstring 53b, MemWrite 53b, REGRET 15f, REGSP 15f, STATMAX, errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, stat(), and sysdbg 79a.

⟨*function* sysfstat 59a⟩≡ (102b)

```
void
sysfstat(void)
{
    byte buf[STATMAX];
    ulong edir;
    int n, fd;

    fd = getmem_w(reg.r[REGSP]+4);
    edir = getmem_w(reg.r[REGSP]+8);
    n = getmem_w(reg.r[REGSP]+12);
    if(sysdbg)
        itrace("fstat(%d, 0x%lux, 0x%lux)", fd, edir, n);

    reg.r[REGRET] = -1;
    if(n > sizeof buf){
        strcpy(errbuf, "stat buffer too big");
        return;
    }
    n = fstat(fd, buf, n);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);
    else
        memio((char*)buf, edir, n, MemWrite);
    reg.r[REGRET] = n;
}
```

Uses MemWrite 53b, REGRET 15f, REGSP 15f, STATMAX, errbuf 54b, errstr(), fstat(), getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, and sysdbg 79a.

7.5 Directory syscalls

⟨*function* syscreate 59b⟩≡ (102b)

```
void
syscreate(void)
{
    char file[1024];
    int n;
    ulong mode, name, perm;

    name = getmem_w(reg.r[REGSP]+4);
    mode = getmem_w(reg.r[REGSP]+8);
    perm = getmem_w(reg.r[REGSP]+12);
    memio(file, name, sizeof(file), MemReadstring);
    if(sysdbg)
        itrace("create(0x%lux='%s', 0x%lux, 0x%lux)", name, file, mode, perm);

    n = create(file, mode, perm);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);

    reg.r[REGRET] = n;
}
```

Uses MemReadstring 53b, REGRET 15f, REGSP 15f, create(), errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, and sysdbg 79a.

⟨*function* sysremove 60a⟩≡ (102b)

```
void
sysremove(void)
{
    char nambuf[1024];
    ulong name;
    int n;

    name = getmem_w(reg.r[REGSP]+4);
    memio(nambuf, name, sizeof(nambuf), MemReadstring);
    if(sysdbg)
        itrace("remove(0x%lux='%s')", name, nambuf);

    n = remove(nambuf);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);
    reg.r[REGRET] = n;
}
```

Uses MemReadstring 53b, REGRET 15f, REGSP 15f, errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, remove(), and sysdbg 79a.

⟨*function* sysfd2path 60b⟩≡ (102b)

```
void
sysfd2path(void)
{
    int n;
    uint fd;
    ulong str;
    char buf[1024];

    fd = getmem_w(reg.r[REGSP]+4);
    str = getmem_w(reg.r[REGSP]+8);
    n = getmem_w(reg.r[REGSP]+12);
    if(sysdbg)
        itrace("fd2path(0x%lux, 0x%lux, 0x%lux)", fd, str, n);
    reg.r[1] = -1;
    if(n > sizeof buf){
        strcpy(errbuf, "buffer too big");
        return;
    }
    n = fd2path(fd, buf, sizeof buf);
    if(n < 0)
        errstr(buf, sizeof buf);
    else
        memio(errbuf, str, n, MemWrite);
    reg.r[REGRET] = n;

}
```

Uses MemWrite 53b, REGRET 15f, REGSP 15f, errbuf 54b, errstr(), fd2path(), getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, and sysdbg 79a.

⟨*function* syschdir 60c⟩≡ (102b)

```
void
syschdir(void)
{
    char file[1024];
    int n;
```

```
    ulong name;

    name = getmem_w(reg.r[REGSP]+4);
    memio(file, name, sizeof(file), MemReadstring);
    if(sysdbg)
        itrace("chdir(0x%lux='%s', 0x%lux)", name, file);

    n = chdir(file);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);

    reg.r[REGRET] = n;
}
```

Uses MemReadstring 53b, REGRET 15f, REGSP 15f, chdir(), errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, and sysdbg 79a.

7.6 Namespace syscalls

⟨*function* sysbind 61a⟩≡ (102b)

```
void
sysbind(void)
{
    ulong pname, pold, flags;
    char name[1024], old[1024];
    int n;

    pname = getmem_w(reg.r[REGSP]+4);
    pold = getmem_w(reg.r[REGSP]+8);
    flags = getmem_w(reg.r[REGSP]+12);
    memio(name, pname, sizeof(name), MemReadstring);
    memio(old, pold, sizeof(old), MemReadstring);
    if(sysdbg)
        itrace("bind(0x%lux='%s', 0x%lux='%s', 0x%lux)", name, name, old, old, flags);

    n = bind(name, old, flags);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);

    reg.r[REGRET] = n;
}
```

Uses MemReadstring 53b, REGRET 15f, REGSP 15f, bind(), errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, and sysdbg 79a.

7.7 Misc syscalls

⟨*function* sysdup 61b⟩≡ (102b)

```
void
sysdup(void)
{
    int oldfd, newfd;
    int n;

    oldfd = getmem_w(reg.r[REGSP]+4);
    newfd = getmem_w(reg.r[REGSP]+8);
    if(sysdbg)
        itrace("dup(%d, %d)", oldfd, newfd);
```

```
    n = dup(oldfd, newfd);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);
    reg.r[REGRET] = n;
}
```

Uses REGRET 15f, REGSP 15f, dup(), errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, reg 15e, and sysdbg 79a.

⟨*constant* OERRLEN 62a⟩≡ (102b)

```
#define OERRLEN 64 /* compatibility; used in _stat etc. */
```

⟨*function* sysexits 62b⟩≡ (102b)

```
void
sysexits(void)
{
    char buf[OERRLEN];
    ulong str;

    str = getmem_w(reg.r[REGSP]+4);
    if(sysdbg)
        itrace("exits(0x%lux)", str);

    // single step to give opportunity to inspect before exit
    count = 1;
    if(str != 0) {
        memio(buf, str, sizeof buf, MemRead);
        Bprint(bout, "exits(%s)\n", buf);
    }
    else
        Bprint(bout, "exits(0)\n");
}
```

Uses MemRead 53b, OERRLEN<> 62a, REGSP 15f, bout 17d, count 23a, getmem_w() 49a, itrace() 92b, memio() 53c, reg 15e, and sysdbg 79a.

⟨*function* syssleep 62c⟩≡ (102b)

```
void
syssleep(void)
{
    ulong len;
    int n;

    len = getmem_w(reg.r[REGSP]+4);
    if(sysdbg)
        itrace("sleep(%d)", len);

    n = sleep(len);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);

    reg.r[REGRET] = n;
}
```

Uses REGRET 15f, REGSP 15f, errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, reg 15e, sleep(), and sysdbg 79a.

⟨*function* syspipe 62d⟩≡ (102b)

```
void
syspipe(void)
{
    int n, p[2];
    ulong fd;
```

```
    fd = getmem_w(reg.r[REGSP]+4);
    if(sysdbg)
        itrace("pipe(%lux)", fd);

    n = pipe(p);
    if(n < 0)
        errstr(errbuf, sizeof errbuf);
    else {
        putmem_w(fd, p[0]);
        putmem_w(fd+4, p[1]);
    }
    reg.r[REGRET] = n;
}
```

Uses REGRET 15f, REGSP 15f, errbuf 54b, errstr(), getmem_w() 49a, itrace() 92b, pipe(), putmem_w() 50d, reg 15e, and sysdbg 79a.

⟨*global* nofunc 63a⟩≡ (102b)

```
ulong nofunc;
```

⟨*function* sysnotify 63b⟩≡ (102b)

```
void
sysnotify(void)
{
    nofunc = getmem_w(reg.r[REGSP]+4);
    if(sysdbg)
        itrace("notify(0x%lux)\n", nofunc);

    reg.r[REGRET] = 0;
}
```

Uses REGRET 15f, REGSP 15f, getmem_w() 49a, itrace() 92b, nofunc 63a, reg 15e, and sysdbg 79a.

7.8 Unsupported syscalls

⟨*function* sysrfork 63c⟩≡ (102b)

```
void
sysrfork(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysexec 63d⟩≡ (102b)

```
void
sysexec(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysawait 63e⟩≡ (102b)

```
void
sysawait(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
```

```
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysswstat 64a⟩≡ (102b)

```
void
sysswstat(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysfwstat 64b⟩≡ (102b)

```
void
sysfwstat(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysnoted 64c⟩≡ (102b)

```
void
sysnoted(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* syssegattach 64d⟩≡ (102b)

```
void
syssegattach(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* syssegdetach 64e⟩≡ (102b)

```
void
syssegdetach(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* syssegfree 64f⟩≡ (102b)

```
void
syssegfree(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* syssegflush 65a⟩≡ (102b)

```
void
syssegflush(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* syssegbrk 65b⟩≡ (102b)

```
void
syssegbrk(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysrendezvous 65c⟩≡ (102b)

```
void
sysrendezvous(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysmount 65d⟩≡ (102b)

```
void
sysmount(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysunmount 65e⟩≡ (102b)

```
void
sysunmount(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysalarm 65f⟩≡ (102b)

```
void
sysalarm(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysfauth 66a⟩≡ (102b)

```
void
sysfauth(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

⟨*function* sysfversion 66b⟩≡ (102b)

```
void
sysfversion(void)
{
    Bprint(bout, "No system call %s\n", sysctab[reg.r[REGARG]]);
    exits(0);
}
```

Uses REGARG 15f, bout 17d, reg 15e, and sysctab.

Chapter 8

Debugger

8.1 Overview

⟨cmd() *locals* 67a⟩+≡ (21c) ◁21b 67b▷

```
static char *cmdlet = ":$?/=>"; //$
```

⟨cmd() *locals* 67b⟩+≡ (21c) ◁67a 67c▷

```
char buf[128];
char addr[128];
```

⟨cmd() *locals* 67c⟩+≡ (21c) ◁67b 67d▷

```
char lastcmd[128];
```

⟨cmd() *locals* 67d⟩+≡ (21c) ◁67c

```
char *a, *cp, *gotint;
int n, i;
```

⟨cmd() *read and parse command and address from user input* 67e⟩≡ (21c)

```
p = buf;
n = 0;

for(;;) {
    i = Bgetc(bin);
    if(i < 0)
        exits(0);
    *p++ = i;
    n++;
    if(i == '\n')
        break;
}

if(buf[0] == '\n')
    strcpy(buf, lastcmd);
else {
    buf[n-1] = '\0';
    strcpy(lastcmd, buf);
}

p = buf;
a = addr;
for(;;) {
    p = nextc(p);
    if(*p == '\0' || strchr(cmdlet, *p))
        break;
    *a++ = *p++;
```

```
}
*a = '\0';

cmdcount = 1;
cp = strchr(addr, ',');
if(cp != nil) {
    if(cp[1] == '#')
        cmdcount = strtoul(cp+2, &gotint, 16);
    else
        cmdcount = strtoul(cp+1, &gotint, 0);
    *cp = '\0';
}
```

Uses bin 17d, cmdcount 85i, nextc() 68a, and nil.

⟨*function* nextc 68a⟩≡ (102a)

```
char*
nextc(char *p)
{
    while(*p && (*p == ' ' || *p == '\t') && *p != '\n')
        p++;

    if(*p == '\n')
        *p = '\0';

    return p;
}
```

⟨cmd() *command cases* 68b⟩+≡ (21c) ◁22a 68c▷

```
case '$': //$
    dollar(p+1);
    break;
```

Uses dollar() 71.

⟨cmd() *command cases* 68c⟩+≡ (21c) ◁68b 68d▷

```
case '/':
case '?':
    dot = expr(addr);
    for(i = 0; i < cmdcount; i++)
        quesie(p+1);
    break;
```

Uses cmdcount 85i, dot 21a, expr() 70a, and quesie() 77a.

⟨cmd() *command cases* 68d⟩+≡ (21c) ◁68c 68e▷

```
case '=':
    eval(addr, p+1);
    break;
```

Uses eval() 76.

⟨cmd() *command cases* 68e⟩+≡ (21c) ◁68d

```
case '>':
    setreg(addr, p+1);
    break;
```

Uses setreg() 77b.

8.2 Interface

⟨*global* fmt 68f⟩≡ (102a)

```
char fmt = 'X';
```

Uses fmt 68f.

⟨*global* width 69a⟩≡ (102a)

```
int width = 60;
```

Uses width 69a.

⟨*global* inc 69b⟩≡ (102a)

```
int inc;
```

⟨*function* reset 69c⟩≡ (102a)

```
void
reset(void)
{
    int i, l, m;
    Segment *s;
    Breakpoint *b;

    memset(&reg, 0, sizeof(Registers));

    for(i = 0; i > Nseg; i++) {
        s = &memory.seg[i];
        l = ((s->end-s->base)/BY2PG)*sizeof(byte*);
        for(m = 0; m < l; m++)
            if(s->table[m])
                free(s->table[m]);
        free(s->table);
    }
    free(iprof);
    memset(&memory, 0, sizeof(memory));

    for(b = bplist; b; b = b->next)
        b->done = b->count;
}
```

Uses BY2PG 18g, Nseg 15g, bplist 82a, iprof 85d, memory 16b, and reg 15e.

⟨*function* numsym 69d⟩≡ (102a)

```
char*
numsym(char *addr, ulong *val)
{
    char tsym[128], *t;
    static char *delim = "`'<>/\\@*|-~+-/=?\n";
    Symbol s;
    char c;

    t = tsym;
    while(c = *addr) {
        if(strchr(delim, c))
            break;
        *t++ = c;
        addr++;
    }
    t[0] = '\0';

    if(strcmp(tsym, ".") == 0) {
        *val = dot;
        return addr;
    }

    if(lookup(0, tsym, &s))
        *val = s.value;
    else {
        if(tsym[0] == '#')
```

```
            *val = strtoul(tsym+1, 0, 16);
        else
            *val = strtoul(tsym, 0, 0);
    }
    return addr;
}
```

Uses dot 21a.

⟨*function* expr 70a⟩≡ (102a)

```
ulong
expr(char *addr)
{
    ulong t, t2;
    char op;

    if(*addr == '\0')
        return dot;

    addr = numsym(addr, &t);

    if(*addr == '\0')
        return t;

    addr = nextc(addr);
    op = *addr++;
    numsym(addr, &t2);
    switch(op) {
    default:
        Bprint(bout, "expr syntax\n");
        return 0;
    case '+':
        t += t2;
        break;
    case '-':
        t -= t2;
        break;
    case '%':
        t /= t2;
        break;
    case '&':
        t &= t2;
        break;
    case '|':
        t |= t2;
        break;
    }

    return t;
}
```

Uses bout 17d, dot 21a, nextc() 68a, and numsym() 69d.

⟨*function* buildargv 70b⟩≡ (102a)

```
int
buildargv(char *str, char **args, int max)
{
    int na = 0;

    while (na < max) {
        while((*str == ' ' || *str == '\t' || *str == '\n') && *str != '\0')
            str++;
```

```
        if(*str == '\0')
            return na;

        args[na++] = str;
        while(!(*str == ' ' || *str == '\t'|| *str == '\n') && *str != '\0')
            str++;

        if(*str == '\n')
            *str = '\0';

        if(*str == '\0')
            break;

        *str++ = '\0';
    }
    return na;
}
```

8.2.1 Inspecting: $

⟨*function* dollar 71⟩≡ (102a)

```
void
dollar(char *cp)
{
    cp = nextc(cp);

    switch(*cp) {
    case 'c':
        stktrace(*cp);
        break;

    case 'C':
        stktrace(*cp);
        break;

    case 'b':
        dobplist();
        break;

    case 'r':
        dumpreg();
        break;

    case 'R':
        dumpreg();

    case 'f':
        dumpfreg();
        break;

    case 'F':
        dumpdreg();
        break;

    case 'q':
        exits(0);
        break;
```

```
case 'Q':
    isum();
    tlbsum();
    segsum();
    break;

case 't':
    cp++;
    switch(*cp) {
    case '\0':
        trace = true;
        break;
    case '0':
        trace = false;
        sysdbg = false;
        calltree = false;
        break;
    case 's':
        sysdbg = true;
        break;
    case 'i':
        trace = true;
        break;
    ⟨dollar() t cases 81b⟩
    default:
        Bprint(bout, "$t[0sic]\n"); //$
        break;
    }
    break;

case 'i':
    cp++;
    switch(*cp) {
    default:
        Bprint(bout, "$i[itsa]\n"); //$
        break;
    case 'i':
        isum();
        break;
    case 't':
        tlbsum();
        break;
    case 's':
        segsum();
        break;
    case 'a':
        isum();
        tlbsum();
        segsum();
        iprofile();
        break;
    case 'p':
        iprofile();
        break;
    }
default:
    Bprint(bout, "?\n");
    break;

}
```

```
}
```

Uses bout 17d, calltree 81a, dobplist() 82d, dumpdreg() 78c, dumpfreg() 78b, dumpreg() 78a, false, iprofile() 88e, isum() 86, nextc() 68a, segsum() 88a, stktrace() 80c, sysdbg 79a, tlbsum() 87a, trace 92a, and true.

8.2.2 Controlling: :

⟨colon() *locals* 73a⟩≡ (22b) 73e ▷

```
char tbuf[512];
```

⟨colon() *print current instruction* 73b⟩≡ (22b)

```
symoff(tbuf, sizeof(tbuf), dot, CTEXT);
Bprint(bout, tbuf);
if(fmt == 'z')
    printsource(dot);
```

Uses CTEXT, bout 17d, dot 21a, fmt 68f, and printsource() 79d.

⟨colon() *command which return cases* 73c⟩≡ (22b) 73d ▷

```
case 'b':
    breakpoint(addr, cp+1);
    return;
```

Uses breakpoint() 82e.

⟨colon() *command which return cases* 73d⟩+≡ (22b) ◁ 73c

```
case 'd':
    delbpt(addr);
    return;
```

Uses delbpt() 83a.

⟨colon() *locals* 73e⟩+≡ (22b) ◁ 73a

```
int argc;
char *argv[100];
```

⟨colon() *command cases* 73f⟩+≡ (22b) ◁ 22c 73g ▷

```
case 'r':
    reset();
    argc = buildargv(cp+1, argv, 100);
    initstk(argc, argv);
    count = 0;
    atbpt = false;
    run();
    break;
```

Uses atbpt 82c, buildargv() 70b, count 23a, false, initstk() 20e, reset() 69c, and run() 23c.

⟨colon() *command cases* 73g⟩+≡ (22b) ◁ 73f

```
case 's':
    cp = nextc(cp+1);
    count = 0;
    if(*cp)
        count = strtoul(cp, 0, 0);
    if(count == 0)
        count = 1;
    atbpt = false;
    run();
    break;
```

Uses atbpt 82c, count 23a, false, nextc() 68a, and run() 23c.

8.3 Format

⟨*function* pfmt 74⟩≡ (102a)

```
int
pfmt(char fmt, int mem, ulong val)
{
    int c, i;
    Symbol s;
    char *p, ch, str[1024];

    c = 0;
    switch(fmt) {
    case 'o':
        c = Bprint(bout, "%-4lo ", mem? (ushort)getmem_2(dot): val);
        inc = 2;
        break;

    case 'O':
        c = Bprint(bout, "%-8lo ", mem? getmem_4(dot): val);
        inc = 4;
        break;

    case 'q':
        c = Bprint(bout, "%-4lo ", mem? (short)getmem_2(dot): val);
        inc = 2;
        break;

    case 'Q':
        c = Bprint(bout, "%-8lo ", mem? (long)getmem_4(dot): val);
        inc = 4;
        break;

    case 'd':
        c = Bprint(bout, "%-5ld ", mem? (short)getmem_2(dot): val);
        inc = 2;
        break;


    case 'D':
        c = Bprint(bout, "%-8ld ", mem? (long)getmem_4(dot): val);
        inc = 4;
        break;

    case 'x':
        c = Bprint(bout, "#%-4lux ", mem? (long)getmem_2(dot): val);
        inc = 2;
        break;

    case 'X':
        c = Bprint(bout, "#%-8lux ", mem? (long)getmem_4(dot): val);
        inc = 4;
        break;

    case 'u':
        c = Bprint(bout, "%-5ld ", mem? (ushort)getmem_2(dot): val);
        inc = 2;
        break;

    case 'U':
        c = Bprint(bout, "%-8ld ", mem? (ulong)getmem_4(dot): val);
```

```
        inc = 4;
        break;

    case 'b':
        c = Bprint(bout, "%-3ld ", mem? getmem_b(dot): val);
        inc = 1;
        break;

    case 'c':
        c = Bprint(bout, "%c ", (int)(mem? getmem_b(dot): val));
        inc = 1;
        break;

    case 'C':
        ch = mem? getmem_b(dot): val;
        if(isprint(ch))
            c = Bprint(bout, "%c ", ch);
        else
            c = Bprint(bout, "\\x%.2x ", ch);
        inc = 1;
        break;

    case 's':
        i = 0;
        while(ch = getmem_b(dot+i))
            str[i++] = ch;
        str[i] = '\0';
        dot += i;
        c = Bprint(bout, "%s", str);
        inc = 0;
        break;

    case 'S':
        i = 0;
        while(ch = getmem_b(dot+i))
            str[i++] = ch;
        str[i] = '\0';
        dot += i;
        for(p = str; *p; p++)
            if(isprint(*p))
                c += Bprint(bout, "%c", *p);
            else
                c += Bprint(bout, "\\x%.2ux", *p);
        inc = 0;
        break;

    case 'Y':
        p = ctime(mem? getmem_b(dot): val);
        p[30] = '\0';
        c = Bprint(bout, "%s", p);
        inc = 4;
        break;

    case 'a':
        symoff(str, sizeof(str), dot, CTEXT);
        c = Bprint(bout, str);
        inc = 0;
        break;

    case 'e':
```

```
        for(i = 0; globalsym(&s, i); i++)
            Bprint(bout, "%-15s #%lux\n", s.name, getmem_4(s.value));
        inc = 0;
        break;

    case 'I':
    case 'i':
        inc = machdata->das(symmap, dot, fmt, str, sizeof(str));
        if(inc < 0) {
            Bprint(bout, "5i: %r\n");
            return 0;
        }
        c = Bprint(bout, "\t%s", str);
        break;

    case 'n':
        c = width+1;
        inc = 0;
        break;

    case '-':
        c = 0;
        inc = -1;
        break;

    case '+':
        c = 0;
        inc = 1;
        break;

    case '^':
        c = 0;
        if(inc > 0)
            inc = -inc;
        break;

    case 'z':
        if(findsym(dot, CTEXT, &s))
            Bprint(bout, "  %s() ", s.name);
        printsource(dot);
        inc = 0;
        break;

    default:
        Bprint(bout, "bad modifier\n");
        return 0;
    }
    return c;
}
```

Uses CTEXT, bout 17d, dot 21a, getmem_2() 49c, getmem_4() 50a, getmem_b() 48e, inc 69b, isprint, printsource() 79d, symmap 18c, and width 69a.

⟨*function* eval 76⟩≡ (102a)

```
void
eval(char *addr, char *p)
{
    ulong val;

    val = expr(addr);
    p = nextc(p);
```

```
    if(*p == '\0') {
        p[0] = fmt;
        p[1] = '\0';
    }
    pfmt(*p, 0, val);
    Bprint(bout, "\n");
}
```

Uses bout 17d, expr() 70a, fmt 68f, nextc() 68a, and pfmt() 74.

⟨*function* quesie 77a⟩≡ (102a)

```
void
quesie(char *p)
{
    int c, count, i;
    char tbuf[512];

    c = 0;
    symoff(tbuf, sizeof(tbuf), dot, CTEXT);
    Bprint(bout, "%s?\t", tbuf);

    while(*p) {
        p = nextc(p);
        if(*p == '"') {
            for(p++; *p && *p != '"'; p++) {
                Bputc(bout, *p);
                c++;
            }
            if(*p)
                p++;
            continue;
        }
        count = 0;
        while(*p >= '0' && *p <= '9')
            count = count*10 + (*p++ - '0');
        if(count == 0)
            count = 1;
        p = nextc(p);
        if(*p == '\0') {
            p[0] = fmt;
            p[1] = '\0';
        }
        for(i = 0; i < count; i++) {
            c += pfmt(*p, 1, 0);
            dot += inc;
            if(c > width) {
                Bprint(bout, "\n");
                symoff(tbuf, sizeof(tbuf), dot, CTEXT);
                Bprint(bout, "%s?\t", tbuf);
                c = 0;
            }
        }
        fmt = *p++;
        p = nextc(p);
    }
    Bprint(bout, "\n");
}
```

Uses CTEXT, bout 17d, dot 21a, fmt 68f, inc 69b, nextc() 68a, pfmt() 74, and width 69a.

⟨*function* setreg 77b⟩≡ (102a)

```
void
```

```
setreg(char *addr, char *cp)
{
    int rn;

    dot = expr(addr);
    cp = nextc(cp);
    if(strcmp(cp, "pc") == 0) {
        reg.r[REGPC] = dot;
        return;
    }
    if(strcmp(cp, "sp") == 0) {
        reg.r[REGSP] = dot;
        return;
    }
    if(*cp++ == 'r') {
        rn = strtoul(cp, 0, 10);
        if(rn > 0 && rn < 16) {
            reg.r[rn] = dot;
            return;
        }
    }
    Bprint(bout, "bad register\n");
}
```

Uses REGPC 15f, REGSP 15f, bout 17d, dot 21a, expr() 70a, nextc() 68a, and reg 15e.

8.4 Dumpers

⟨*function* dumpreg 78a⟩≡ (98a)

```
void
dumpreg(void)
{
    int i;

    Bprint(bout, "PC  #%-8lux SP  #%-8lux \n",
                reg.r[REGPC], reg.r[REGSP]);

    for(i = 0; i < 16; i++) {
        if((i%4) == 0 && i != 0)
            Bprint(bout, "\n");
        Bprint(bout, "R%-2d #%-8lux ", i, reg.r[i]);
    }
    Bprint(bout, "\n");
}
```

Uses REGPC 15f, REGSP 15f, bout 17d, and reg 15e.

⟨*function* dumpfreg 78b⟩≡ (98a)

```
void
dumpfreg(void)
{
}
```

⟨*function* dumpdreg 78c⟩≡ (98a)

```
void
dumpdreg(void)
{
}
```

8.5 Traces

8.5.1 Syscalls trace

⟨*global* sysdbg 79a⟩≡ (97b)

```
bool sysdbg;
```

⟨sysnop *strace* 79b⟩≡ (54a)

```
if(sysdbg)
    itrace("nop()");
```

Uses itrace() 92b and sysdbg 79a.

8.5.2 Stack trace

⟨*constant* STRINGSZ 79c⟩≡ (99b)

```
#define STRINGSZ 128
```

⟨*function* printsource 79d⟩≡ (99b)

```
/*
 * print the value of dot as file:line
 */
void
printsource(long dot)
{
    char str[STRINGSZ];

    if (fileline(str, STRINGSZ, dot))
        Bprint(bout, "%s", str);
}
```

Uses STRINGSZ<> 79c and bout 17d.

⟨*function* printlocals 79e⟩≡ (99b)

```
void
printlocals(Symbol *fn, ulong fp)
{
    int i;
    Symbol s;

    s = *fn;
    for (i = 0; localsym(&s, i); i++) {
        if (s.class != CAUTO)
            continue;
        Bprint(bout, "\t%s=#%lux\n", s.name, getmem_4(fp-s.value));
    }
}
```

Uses CAUTO, bout 17d, and getmem_4() 50a.

⟨*function* printparams 79f⟩≡ (99b)

```
void
printparams(Symbol *fn, ulong fp)
{
    int i;
    Symbol s;
    int first;

    fp += mach->szreg;   /* skip saved pc */
    s = *fn;
    for (first = i = 0; localsym(&s, i); i++) {
```

```
        if (s.class != CPARAM)
            continue;
        if (first++)
            Bprint(bout, ", ");
        Bprint(bout, "%s=#%lux", s.name, getmem_4(fp+s.value));
    }
    Bprint(bout, ") ");
}
```

Uses CPARAM, bout 17d, and getmem_4() 50a.

⟨*constant* STARTSYM 80a⟩≡ (99b)

```
#define STARTSYM "_main"
```

⟨*constant* FRAMENAME 80b⟩≡ (99b)

```
#define FRAMENAME ".frame"
```

⟨*function* stktrace 80c⟩≡ (99b)

```
void
stktrace(int modif)
{
    ulong pc, sp;
    Symbol s, f;
    int i;
    char buf[512];

    pc = reg.r[REGPC];
    sp = reg.r[REGSP];
    i = 0;
    while (findsym(pc, CTEXT, &s)) {
        if(strcmp(STARTSYM, s.name) == 0) {
            Bprint(bout, "%s() at #%llux\n", s.name, s.value);
            break;
        }
        if (pc == s.value) /* at first instruction */
            f.value = 0;
        else if (findlocal(&s, FRAMENAME, &f) == 0)
            break;
        if (s.type == 'L' || s.type == 'l' || pc <= s.value+4)
            pc = reg.r[REGLINK];
        else pc = getmem_4(sp);
        sp += f.value;
        Bprint(bout, "%s(", s.name);
        printparams(&s, sp);
        printsource(s.value);
        Bprint(bout, " called from ");
        symoff(buf, sizeof(buf), pc-8, CTEXT);
        Bprint(bout, buf);
        printsource(pc-8);
        Bprint(bout, "\n");
        if(modif == 'C')
            printlocals(&s, sp);
        if(++i > 40){
            Bprint(bout, "(trace truncated)\n");
            break;
        }
    }
}
```

Uses CTEXT, FRAMENAME<> 80b, REGLINK 15f, REGPC 15f, REGSP 15f, STARTSYM<> 80a, bout 17d, getmem_4() 50a, printlocals() 79e, printparams() 79f, printsource() 79d, and reg 15e.

8.5.3 Call tree trace

⟨*global* `calltree` 81a⟩≡ (97b)
```
bool calltree;
```

⟨`dollar()` *t cases* 81b⟩≡ (71)
```
case 'c':
    calltree = true;
    break;
```
Uses `calltree` 81a and `true`.

⟨`dpex()` *if calltree, when add operation* 81c⟩≡ (28f)
```
if(calltree && rd == REGPC && o2 == 0) {
    Symbol s;

    findsym(o1 + o2, CTEXT, &s);
    Bprint(bout, "%8lux return to %lux %s r0=%lux\n",
                reg.r[REGPC], o1 + o2, s.name, reg.r[REGRET]);
}
```
Uses `CTEXT`, `REGPC` 15f, `REGRET` 15f, `bout` 17d, `calltree` 81a, and `reg` 15e.

⟨`Ibl()` *if calltree* 81d⟩≡ (41d)
```
if(calltree) {
    findsym(v, CTEXT, &s);
    Bprint(bout, "%8lux %s(", reg.r[REGPC], s.name);
    printparams(&s, reg.r[REGSP]);
    Bprint(bout, "from ");
    printsource(reg.r[REGPC]);
    Bputc(bout, '\n');
}
```
Uses `CTEXT`, `REGPC` 15f, `REGSP` 15f, `bout` 17d, `calltree` 81a, `printparams()` 79f, `printsource()` 79d, and `reg` 15e.

8.6 Breakpoints

⟨*enum* `breakpoint_kind` 81e⟩≡ (95)
```
enum breakpoint_kind
{
    Instruction  = 1,

    Read  = 2,
    Write  = 4,
    Access  = Read|Write,

    Equal  = 4|8,
};
```
Uses `Read` 81e and `Write` 81e.

⟨*struct* `Breakpoint` 81f⟩≡ (95)
```
struct Breakpoint
{
    //enum<breakpoint_kind>
    int  type;  /* Instruction/Read/Access/Write/Equal */

    uintptr addr;  /* Place at address */
    int  count;  /* To execute count times or value */
    int  done;  /* How many times passed through */

    // Extra
    ⟨Breakpoint extra fields 82b⟩
};
```

⟨*global* bplist 82a⟩≡ (97b)

```
// list<Breakpoint> (next = Breakpoint.next)
Breakpoint *bplist;
```

⟨Breakpoint *extra fields* 82b⟩≡ (81f)

```
Breakpoint* next;  /* Link to next one */
```

⟨*global* atbpt 82c⟩≡ (97b)

```
bool atbpt;
```

⟨*function* dobplist 82d⟩≡ (98b)

```
void
dobplist(void)
{
    Breakpoint *b;
    char buf[512];

    for(b = bplist; b; b = b->next) {
        switch(b->type) {
        case Instruction:
            Bprint(bout, "0x%lux,%d:b %d done, at ", b->addr, b->count, b->done);
            symoff(buf, sizeof(buf), b->addr, CTEXT);
            Bprint(bout, buf);
            break;

        case Access:
            Bprint(bout, "0x%lux,%d:ba %d done, at ", b->addr, b->count, b->done);
            symoff(buf, sizeof(buf), b->addr, CDATA);
            Bprint(bout, buf);
            break;

        case Read:
            Bprint(bout, "0x%lux,%d:br %d done, at ", b->addr, b->count, b->done);
            symoff(buf, sizeof(buf), b->addr, CDATA);
            Bprint(bout, buf);
            break;

        case Write:
            Bprint(bout, "0x%lux,%d:bw %d done, at ", b->addr, b->count, b->done);
            symoff(buf, sizeof(buf), b->addr, CDATA);
            Bprint(bout, buf);
            break;

        case Equal:
            Bprint(bout, "0x%lux,%d:be at ", b->addr, b->count);
            symoff(buf, sizeof(buf), b->addr, CDATA);
            Bprint(bout, buf);
            break;
        }
        Bprint(bout, "\n");
    }
}
```

Uses Access 81e, CDATA, CTEXT, Equal 81e, Instruction 81e, Read 81e, Write 81e, bout 17d, and bplist 82a.

⟨*function* breakpoint 82e⟩≡ (98b)

```
void
breakpoint(char *addr, char *cp)
{
    Breakpoint *b;
    int type;
```

```
    cp = nextc(cp);
    type = Instruction;

    switch(*cp) {
    case 'r':
        membpt++;
        type = Read;
        break;
    case 'a':
        membpt++;
        type = Access;
        break;
    case 'w':
        membpt++;
        type = Write;
        break;
    case 'e':
        membpt++;
        type = Equal;
        break;
    }
    b = emalloc(sizeof(Breakpoint));
    b->addr = expr(addr);
    b->type = type;
    b->count = cmdcount;
    b->done = cmdcount;

    b->next = bplist;
    bplist = b;
}
```

Uses Access 81e, Equal 81e, Instruction 81e, Read 81e, Write 81e, bplist 82a, cmdcount 85i, emalloc() 94a, expr() 70a, membpt 84b, and nextc() 68a.

⟨*function* delbpt 83a⟩≡ (98b)

```
void
delbpt(char *addr)
{
    Breakpoint *b, **l;
    ulong baddr;

    baddr = expr(addr);
    l = &bplist;
    for(b = *l; b; b = b->next) {
        if(b->addr == baddr) {
            if(b->type != Instruction)
                membpt++;
            *l = b->next;
            free(b);
            return;
        }
        l = &b->next;
    }

    Bprint(bout, "no breakpoint\n");
}
```

Uses Instruction 81e, bout 17d, bplist 82a, expr() 70a, and membpt 84b.

⟨*function* brkchk 83b⟩≡ (98b)

```
void
```

```
brkchk(ulong addr, int type)
{
    Breakpoint *b;

    for(b = bplist; b; b = b->next) {
        if(b->addr == addr && (b->type&type)) {
            if(b->type == Equal && getmem_4(addr) == b->count) {
                count = 1;
                atbpt = true;
                return;
            }
            if(--b->done == 0) {
                b->done = b->count;
                count = 1;
                atbpt = true;
                return;
            }
        }
    }
}
```

Uses `Equal` 81e, `atbpt` 82c, `bplist` 82a, `count` 23a, `getmem_4()` 50a, and `true`.

8.6.1 Code breakpoint

⟨`run()` *check for breakpoints* 84a⟩≡ (23c)

```
if(bplist)
    brkchk(reg.r[REGPC], Instruction);
```

Uses `Instruction` 81e, `REGPC` 15f, `bplist` 82a, `brkchk()` 83b, and `reg` 15e.

8.6.2 Memory breakpoint

⟨*global* `membpt` 84b⟩≡ (97b)

```
bool membpt;
```

⟨`getmem_x()` *if membpt* 84c⟩≡ (49a 48)

```
if(membpt)
    brkchk(addr, Read);
```

Uses `Read` 81e, `brkchk()` 83b, and `membpt` 84b.

⟨`putmem_x()` *if membpt* 84d⟩≡ (50)

```
if(membpt)
    brkchk(addr, Write);
```

Uses `Write` 81e, `brkchk()` 83b, and `membpt` 84b.

Chapter 9

Profiler

⟨`Inst` *profiling fields* 85a⟩≡ (15a)

```
// profiling info
int count;
int taken;
int useddelay;
```

⟨`run()` *profile current instruction class* 85b⟩≡ (23c)

```
// profiling
reg.ip->count++;
```

Uses `reg` 15e.

⟨*function* `Percent` 85c⟩≡ (101b)

```
#define Percent(num, max) ((max)?((num)*100)/(max):0)
```

⟨*global* `iprof` 85d⟩≡ (97b)

```
ulong *iprof;
```

⟨*constant* `PROFGRAN` 85e⟩≡ (18g)

```
PROFGRAN = 4,
```

⟨`initmemory()` *Text segment initilisation* 85f⟩+≡ (19a) ◁19e

⟨`initmemory()` *iprof allocation* 85g⟩

⟨`initmemory()` *iprof allocation* 85g⟩≡ (85f)

```
iprof = emalloc(((s->end - s->base)/PROFGRAN)*sizeof(long));
```

Uses `PROFGRAN` 85e, `emalloc()` 94a, and `iprof` 85d.

⟨*global* `tables` 85h⟩≡ (101b)

```
Inst *tables[] = { itab, 0 };
```

Uses `itab` 15b and `tables` 85h.

⟨*global* `cmdcount` 85i⟩≡ (97b)

```
int cmdcount;
```

⟨*global* `nopcount` 85j⟩≡ (97b)

```
int nopcount;
```

⟨*function* isum 86⟩≡ (101b)

```
void
isum(void)
{
    Inst *i;
    int total, mems, arith, branch;
    int useddelay, taken, syscall;
    int pct, j;

    total = 0;
    mems = 0;
    arith = 0;
    branch = 0;
    useddelay = 0;
    taken = 0;
    syscall = 0;

    /* Compute the total so we can have percentages */
    for(i = itab; i->func; i++)
        if(i->name && i->count)
            total += i->count;

    Bprint(bout, "\nInstruction summary.\n\n");

    for(j = 0; tables[j]; j++) {
        for(i = tables[j]; i->func; i++) {
            if(i->name) {
                /* This is gross */
                if(i->count == 0)
                    continue;
                pct = Percent(i->count, total);
                if(pct != 0)
                    Bprint(bout, "%-8ud %3d%% %s\n",
                        i->count, Percent(i->count,
                        total), i->name);
                else
                    Bprint(bout, "%-8ud      %s\n",
                        i->count, i->name);


                switch(i->type) {
                case Imem:
                    mems += i->count;
                    break;
                case Iarith:
                    arith += i->count;
                    break;
                case Ibranch:
                    branch += i->count;
                    taken += i->taken;
                    useddelay += i->useddelay;
                    break;
                case Isyscall:
                    syscall += i->count;
                    break;
                default:
                    fatal(false, "isum bad stype %d\n", i->type);
                }

            }
```

```
        }
    }

    Bprint(bout, "\n%-8ud      Memory cycles\n", mems+total);
    Bprint(bout, "%-8ud %3d%% Instruction cycles\n",
            total, Percent(total, mems+total));
    Bprint(bout, "%-8ud %3d%% Data cycles\n\n",
            mems, Percent(mems, mems+total));

    Bprint(bout, "%-8ud %3d%% Arithmetic\n",
            arith, Percent(arith, total));

    Bprint(bout, "%-8ud %3d%% System calls\n",
            syscall, Percent(syscall, total));

    Bprint(bout, "%-8ud %3d%% Branches\n",
            branch, Percent(branch, total));

    Bprint(bout, "   %-8ud %3d%% Branches taken\n",
            taken, Percent(taken, branch));

    Bprint(bout, "   %-8ud %3d%% Delay slots\n",
            useddelay, Percent(useddelay, branch));

    Bprint(bout, "   %-8ud %3d%% Unused delay slots\n",
            branch-useddelay, Percent(branch-useddelay, branch));

    Bprint(bout, "%-8ud %3d%% Program total delay slots\n",
            nopcount, Percent(nopcount, total));
}
```

Uses Iarith 14c, Ibranch 14c, Imem 14c, Isyscall 14c, Percent<> 85c, bout 17d, false, fatal() 93a, itab 15b, nopcount 85j, and tables 85h.

⟨*function* tlbsum 87a⟩≡ (101b)

```
void
tlbsum(void)
{
    if(tlb.on == false)
        return;

    Bprint(bout, "\n\nTlb summary\n");

    Bprint(bout, "\n%-8d User entries\n", tlb.tlbsize);
    Bprint(bout, "%-8d Accesses\n", tlb.hit+tlb.miss);
    Bprint(bout, "%-8d Tlb hits\n", tlb.hit);
    Bprint(bout, "%-8d Tlb misses\n", tlb.miss);
    Bprint(bout, "%7d%% Hit rate\n", Percent(tlb.hit, tlb.hit+tlb.miss));
}
```

Uses Percent<> 85c, bout 17d, false, and tlb 46f.

⟨*global* stype 87b⟩≡ (101b)

```
char *stype[] = { "Stack", "Text", "Data", "Bss" };
```

Uses stype 87b.

⟨Segment *profiling fields* 87c⟩≡ (16c)

```
int rss;
int refs;
```

⟨*function* segsum 88a⟩≡ (101b)

```
void
segsum(void)
{
    Segment *s;
    int i;

    Bprint(bout, "\n\nMemory Summary\n\n");
    Bprint(bout, "      Base     End      Resident References\n");
    for(i = 0; i < Nseg; i++) {
        s = &memory.seg[i];
        Bprint(bout, "%-5s %.8lux %.8lux %-8d %-8d\n",
                stype[i], s->base, s->end, s->rss*BY2PG, s->refs);
    }
}
```

Uses BY2PG 18g, Nseg 15g, bout 17d, memory 16b, and stype 87b.

⟨*struct* Prof 88b⟩≡ (101b)

```
struct Prof
{
    Symbol s;
    long count;
};
```

⟨*global* aprof 88c⟩≡ (101b)

```
// can't use prof, conflict with libc.h prof()
Prof aprof[5000];
```

⟨*function* profcmp 88d⟩≡ (101b)

```
int
profcmp(void *va, void *vb)
{
    Prof *a, *b;

    a = va;
    b = vb;
    return b->count - a->count;
}
```

⟨*function* iprofile 88e⟩≡ (101b)

```
void
iprofile(void)
{
    Prof *p, *n;
    int i, b, e;
    ulong total;

    i = 0;
    p = aprof;
    if(textsym(&p->s, i) == 0)
        return;
    i++;
    for(;;) {
        n = p+1;
        if(textsym(&n->s, i) == 0)
            break;
        b = (p->s.value-textbase)/PROFGRAN;
        e = (n->s.value-textbase)/PROFGRAN;
        while(b < e)
            p->count += iprof[b++];
```

```
        i++;
        p = n;
    }

    qsort(prof, i, sizeof(Prof), profcmp);

    total = 0;
    for(b = 0; b < i; b++)
        total += aprof[b].count;

    Bprint(bout, "  cycles     %% symbol          file\n");
    for(b = 0; b < i; b++) {
        if(aprof[b].count == 0)
            continue;

        Bprint(bout, "%8ld %3ld.%ld %-15s ",
            aprof[b].count,
            100*aprof[b].count/total,
            (1000*aprof[b].count/total)%10,
            aprof[b].s.name);

        printsource(aprof[b].s.value);
        Bputc(bout, '\n');
    }
    memset(prof, 0, sizeof(Prof)*i);
}
```

Uses PROFGRAN 85e, aprof 88c, bout 17d, iprof 85d, printsource() 79d, profcmp() 88d, and textbase 19d.

Chapter 10

Advanced Topics

10.1 System instructions

10.2 Coprocessor instructions

10.3 Float instructions

10.4 Signals

⟨cmd() *initialisation* 90a⟩≡ (21c) 93c ▷

```
notify(catcher);
```

Uses catcher() 90b and notify().

⟨*function* catcher 90b⟩≡ (102a)

```
void
catcher(void *a, char *msg)
{
    static int hit = 0;

    hit++;
    if(hit > 5)
        exits(0);
    USED(a);
    if(strcmp(msg, "interrupt") != 0)
        noted(NDFLT);

    count = 1;
    print("5i\n");
    noted(NCONT);
}
```

Uses NCONT, NDFLT, count 23a, and noted().

10.5 Optimisations

Chapter 11

Conclusion

Appendix A

Debugging

⟨*global* trace 92a⟩≡ (97b)

```
bool trace;
```

⟨*function* itrace 92b⟩≡ (98a)

```
void
itrace(char *fmt, ...)
{
    char buf[128];
    va_list arg;

    va_start(arg, fmt);
    vseprint(buf, buf+sizeof(buf), fmt, arg);
    va_end(arg);

    Bprint(bout, "%8lux %.8lux %2d %s\n",
                    reg.ar, reg.instr, reg.instr_opcode, buf);
    Bflush(bout);
}
```

Uses bout 17d and reg 15e.

⟨*global* shtype 92c⟩≡ (99c)

```
static char* shtype[4] =
{
    "<<",
    ">>",
    "->",
    "@>",
};
```

Uses shtype<> 92c.

⟨*global* cond 92d⟩≡ (99c)

```
static char* cond[16] =
{
    ".EQ", ".NE", ".HS", ".LO",
    ".MI", ".PL", ".VS", ".VC",
    ".HI", ".LS", ".GE", ".LT",
    ".GT", ".LE", "", ".NO",
};
```

Uses cond<> 92d.

Appendix B

Error Management

⟨*function* fatal 93a⟩≡ (98a)

```
void
fatal(bool syserr, char *fmt, ...)
{
    char buf[ERRMAX], *s;
    va_list arg;

    va_start(arg, fmt);
    vseprint(buf, buf+sizeof(buf), fmt, arg);
    va_end(arg);

    s = "5i: %s\n";
    if(syserr)
        s = "5i: %s: %r\n";
    fprint(STDERR, s, buf);
    exits(buf);
}
```

Uses ERRMAX and STDERR.

⟨*global* errjmp 93b⟩≡ (97b)

```
jmp_buf errjmp;
```

⟨cmd() *initialisation* 93c⟩+≡ (21c) ◁90a

```
setjmp(errjmp);
```

Uses errjmp 93b.

⟨*function* undef 93d⟩≡ (99c)

```
void
undef(instruction inst)
{
    Bprint(bout, "undefined instruction trap pc #%lux inst %.8lux op %d\n",
        reg.r[REGPC], inst, reg.instr_opcode);
    longjmp(errjmp, 0);
}
```

Uses REGPC 15f, bout 17d, errjmp 93b, and reg 15e.

Appendix C

Utilities

C.1 Memory Management

⟨*function* `emalloc` 94a⟩≡ (98a)

```
void *
emalloc(ulong size)
{
    void *a;

    a = malloc(size);
    if(a == nil)
        fatal(false, "no memory");

    memset(a, 0, size); //!!
    return a;
}
```

Uses `false`, `fatal()` 93a, and `nil`.

⟨*function* `erealloc` 94b⟩≡ (98a)

```
void *
erealloc(void *a, ulong oldsize, ulong size)
{
    void *n;

    n = malloc(size);
    if(n == nil)
        fatal(false, "no memory");
    memset(n, 0, size);
    if(size > oldsize)
        size = oldsize;
    memmove(n, a, size);
    return n;
}
```

Uses `false`, `fatal()` 93a, and `nil`.

Appendix D

Extra Code

D.1 machine/5i/

D.1.1 machine/5i/arm.h

⟨machine/5i/arm.h 95⟩≡

```
/*
 * arm.h
 */

// forward decls
typedef struct Registers Registers;
typedef struct Segment  Segment;
typedef struct Memory  Memory;
typedef struct Inst  Inst;
typedef struct Icache  Icache;
typedef struct Tlb   Tlb;
typedef struct Breakpoint Breakpoint;

⟨typedef instruction 14a⟩

⟨enum breakpoint_kind 81e⟩

⟨struct Breakpoint 81f⟩

⟨enum ixxx 14c⟩

// added by pad
⟨enum opcode 14b⟩

⟨constant Nmaxtlb 46e⟩

⟨enum regxxx 15f⟩

⟨struct Tlb 46d⟩

⟨struct Icache 48a⟩


⟨struct Inst 15a⟩

⟨struct Registers 15d⟩

⟨enum memxxx 53b⟩
```

⟨*enum* compare_op 42e⟩

⟨*enum* segment_kind 15g⟩

⟨*struct* Segment 16c⟩

⟨*struct* Memory 16a⟩

```
// for cmd.c
void  run(void);
// for cmd.c reset
void  initstk(int, char**);
// for syscalls.c
char*  memio(char*, ulong, int, int);
// for run.c
void  Ssyscall(ulong);
// for run.c
ulong  ifetch(ulong);
// used to be in libmach/, but I copy pasted it in run.c
//int arm_class(instruction);

void  updateicache(ulong addr);

ulong  getmem_2(ulong);
ulong  getmem_4(ulong);
uchar  getmem_b(ulong);
ushort  getmem_h(ulong);
uvlong  getmem_v(ulong);
ulong  getmem_w(ulong);
void  putmem_b(ulong, uchar);
void  putmem_h(ulong, ushort);
void  putmem_v(ulong, uvlong);
void  putmem_w(ulong, ulong);

void  cmd(void);
ulong  expr(char*);
char*  nextc(char*);


void  breakpoint(char*, char*);
void  delbpt(char*);
void  brkchk(ulong, int);
void  dobplist(void);

void  dumpdreg(void);
void  dumpfreg(void);
void  dumpreg(void);
void  stktrace(int);
void  printparams(Symbol*, ulong);
void  printsource(long);

// profiling
void  iprofile(void);
void  isum(void);
void  segsum(void);
void  tlbsum(void);

void*  emalloc(ulong);
void*  erealloc(void*, ulong, ulong);
```

```
void  fatal(int, char*, ...);
void  itrace(char*, ...);

// from libc.h
//long  lnrand(long);

/* Globals */
extern Registers reg;
extern Memory  memory;
extern Icache  icache;
extern Tlb   tlb;

extern Inst  itab[];

extern instruction dot;
extern int  count;

extern Biobuf*  bout;
extern Biobuf*  bin;
extern int  text;
extern ulong textbase;
extern int  datasize;

extern Map* symmap;

extern bool trace;
extern bool sysdbg;
extern bool calltree;
extern Breakpoint* bplist;
extern int  atbpt;
extern int  membpt;

extern jmp_buf errjmp;

extern int  cmdcount;
extern int  nopcount;
extern ulong*  iprof;
```

⟨*enum* `_anon_ (machine/5i/arm.h)` 7 18g⟩

Uses `Breakpoint` 81f, `Icache` 48a, `Inst` 15a, `Memory` 16a, `Registers` 15d, `Segment` 16c, and `Tlb` 46d.

D.1.2 machine/5i/icache.c

⟨`machine/5i/icache.c` 97a⟩≡

```
⟨basic includes 12⟩

⟨function updateicache 48d⟩
```

D.1.3 machine/5i/globals.c

⟨`machine/5i/globals.c` 97b⟩≡

```
⟨basic includes 12⟩

#include <tos.h>

//in run.c
//Inst itab[];
```

⟨*global* reg 15e⟩
⟨*global* memory 16b⟩
⟨*global* text 17b⟩
⟨*global* trace 92a⟩
⟨*global* sysdbg 79a⟩
⟨*global* calltree 81a⟩
⟨*global* icache 48b⟩
⟨*global* tlb 46f⟩
⟨*global* count 23a⟩
⟨*global* errjmp 93b⟩
⟨*global* bplist 82a⟩
⟨*global* atbpt 82c⟩
⟨*global* membpt 84b⟩
⟨*global* cmdcount 85i⟩
⟨*global* nopcount 85j⟩
⟨*global* dot 21a⟩
⟨*global* bixxx 17d⟩
⟨*global* iprof 85d⟩
⟨*global* symmap 18c⟩
⟨*global* datasize 20a⟩
⟨*global* textbase 19d⟩

D.1.4 machine/5i/utils.c

⟨machine/5i/utils.c 98a⟩≡
⟨*basic includes* 12⟩

⟨*function* fatal 93a⟩

⟨*function* itrace 92b⟩

⟨*function* dumpreg 78a⟩

⟨*function* dumpfreg 78b⟩

⟨*function* dumpdreg 78c⟩

⟨*function* emalloc 94a⟩

⟨*function* erealloc 94b⟩

D.1.5 machine/5i/bpt.c

⟨machine/5i/bpt.c 98b⟩≡
⟨*basic includes* 12⟩

```
#include <ctype.h>
```

⟨*function* dobplist 82d⟩

⟨*function* breakpoint 82e⟩

⟨*function* delbpt 83a⟩

⟨*function* brkchk 83b⟩

D.1.6 machine/5i/mem.c

⟨machine/5i/mem.c 99a⟩≡
⟨*basic includes* 12⟩

```
void*  page_of_vaddr(ulong);
```

⟨*function* ifetch 47d⟩

⟨*function* getmem_4 50a⟩

⟨*function* getmem_2 49c⟩

⟨*function* getmem_w 49a⟩

⟨*function* getmem_h 48f⟩

⟨*function* getmem_b 48e⟩

⟨*function* getmem_v 49b⟩

⟨*function* putmem_h 50c⟩

⟨*function* putmem_w 50d⟩

⟨*function* putmem_b 50b⟩

⟨*function* putmem_v 51⟩

⟨*function* memio 53c⟩

⟨*function* dotlb 47c⟩

⟨*function* page_of_vaddr 45⟩

D.1.7 machine/5i/symbols.c

⟨machine/5i/symbols.c 99b⟩≡
⟨*basic includes* 12⟩

⟨*constant* STRINGSZ 79c⟩

⟨*function* printsource 79d⟩

⟨*function* printlocals 79e⟩

⟨*function* printparams 79f⟩

⟨*constant* STARTSYM 80a⟩
⟨*constant* FRAMENAME 80b⟩

⟨*function* stktrace 80c⟩

D.1.8 machine/5i/run.c

⟨machine/5i/run.c 99c⟩≡
⟨*basic includes* 12⟩

⟨*macro* XCAST 29a⟩

```
// forward decl
void undef(ulong);

void Idp0(ulong);
void Idp1(ulong);
void Idp2(ulong);
void Idp3(ulong);

void Imul(ulong);
void Imula(ulong);
void Imull(ulong);

void Iswap(ulong);
void Imem1(ulong);
void Imem2(ulong);
void Ilsm(ulong inst);

void Ib(ulong);
void Ibl(ulong);

int arm_class(instruction w);

//static int dummy;
```

⟨*global* shtype 92c⟩
⟨*global* cond 92d⟩

⟨*global* itab 15b⟩

⟨*function* runcmp 43a⟩

⟨*function* runteq 43c⟩

⟨*function* runtst 43d⟩

⟨*function* run 23c⟩

⟨*function* undef 93d⟩

⟨*function* arm_class 24a⟩

⟨*function* shift 30d⟩

⟨*function* dpex 27e⟩

⟨*function* Idp0 27a⟩

⟨*function* Idp1 29c⟩

⟨*function* Idp2 30b⟩

⟨*function* Idp3 32d⟩

⟨*function* Imul 26a⟩

⟨*function* Imull 33d⟩

⟨*function* Imula 26b⟩

⟨*function* Iswap 35h⟩

⟨*function* Imem1 36c⟩

⟨*function* Imem2 37b⟩

⟨*function* Ilsm 39e⟩

⟨*function* Ib 41b⟩

⟨*function* Ibl 41d⟩

D.1.9 machine/5i/5i.c

⟨machine/5i/5i.c 101a⟩≡
⟨*basic includes* 12⟩

```
#include <tos.h>
```

⟨*global* file 17a⟩
⟨*global* bxxx 17c⟩
⟨*global* fhdr 18a⟩

⟨*function* initmemory 19a⟩

⟨*function* inithdr 18b⟩

⟨*function* initstk 20e⟩

⟨*function* main 17e⟩

D.1.10 machine/5i/stats.c

⟨machine/5i/stats.c 101b⟩≡
⟨*basic includes* 12⟩

⟨*function* Percent 85c⟩

```
typedef struct Prof Prof;
```

⟨*global* tables 85h⟩

⟨*function* isum 86⟩

⟨*function* tlbsum 87a⟩

⟨*global* stype 87b⟩

⟨*function* segsum 88a⟩

⟨*struct* Prof 88b⟩
⟨*global* aprof 88c⟩

⟨*function* profcmp 88d⟩

⟨*function* iprofile 88e⟩

Uses Prof 88b.

D.1.11 machine/5i/cmd.c

⟨machine/5i/cmd.c 102a⟩≡

```
⟨basic includes 12⟩

#include <ctype.h>

⟨global fmt 68f⟩
⟨global width 69a⟩
⟨global inc 69b⟩

⟨function reset 69c⟩

⟨function nextc 68a⟩

⟨function numsym 69d⟩

⟨function expr 70a⟩

⟨function buildargv 70b⟩

⟨function colon 22b⟩

⟨function dollar 71⟩

⟨function pfmt 74⟩

⟨function eval 76⟩

⟨function quesie 77a⟩

⟨function catcher 90b⟩

⟨function setreg 77b⟩

⟨function cmd 21c⟩
```

D.1.12 machine/5i/syscall.c

⟨machine/5i/syscall.c 102b⟩≡

```
⟨basic includes 12⟩

//#define ODIRLEN 116 /* compatibility; used in _stat etc. */
⟨constant OERRLEN 62a⟩

⟨global errbuf 54b⟩
⟨global nofunc 63a⟩

#include "../../lib_core/libc/9syscall/sys.h"

⟨global sysctab ??⟩

⟨function sysnop 54a⟩

⟨function syserrstr 54c⟩
⟨function sysbind 61a⟩
```

⟨*function* sysfd2path 60b⟩

⟨*function* syschdir 60c⟩

⟨*function* sysclose 56a⟩

⟨*function* sysdup 61b⟩

⟨*function* sysexits 62b⟩

⟨*function* sysopen 55b⟩

⟨*function* sysread 56b⟩

⟨*function* syspread 57a⟩

⟨*function* sysseek 57b⟩

⟨*function* syssleep 62c⟩

⟨*function* sysstat 58c⟩

⟨*function* sysfstat 59a⟩

⟨*function* syswrite 58a⟩

⟨*function* syspwrite 58b⟩

⟨*function* syspipe 62d⟩

⟨*function* syscreate 59b⟩

⟨*function* sysbrk 55a⟩

⟨*function* sysremove 60a⟩

⟨*function* sysnotify 63b⟩

⟨*function* sysawait 63e⟩
⟨*function* sysrfork 63c⟩
⟨*function* syswstat 64a⟩
⟨*function* sysfwstat 64b⟩
⟨*function* sysnoted 64c⟩
⟨*function* syssegattach 64d⟩
⟨*function* syssegdetach 64e⟩
⟨*function* syssegfree 64f⟩
⟨*function* syssegflush 65a⟩
⟨*function* sysrendezvous 65c⟩
⟨*function* sysunmount 65e⟩

⟨*function* syssegbrk 65b⟩
⟨*function* sysmount 65d⟩
⟨*function* sysalarm 65f⟩
⟨*function* sysexec 63d⟩

⟨*function* sysfauth 66a⟩

⟨*function* `sysfversion` 66b⟩

⟨*global* `systab` 52⟩

⟨*function* `Ssyscall` 53a⟩

Glossary

```
RISC = Reduced Instruction Set Computer
CISC = Complex Instruction Set Computer
ARM  = Akorn Risc Machines
ISA  = Instruction Set Architecture
```

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Access: 81e, 82d, 82e
ALARM: 52
aprof: 88c, 88e
arm_class(): 23c, 24a
atbpt: 22b, 22c, 73f, 73g, 82c, 83b
AWAIT: 52
bi: 17c, 17e
bin: 17d, 17e, 56b, 67e
BIND: 52
bind(): 61a
bo: 17c, 17e
bout: 17d, 17e, 21c, 22b, 42f, 43a, 43c, 43d, 45, 47d, 50c, 50d, 53a, 53c, 54a, 58a, 62b, 63c, 63d, 63e, 64a, 64b, 64c, 64d, 64e, 64f, 65a, 65b, 65c, 65d, 65e, 65f, 66a, 66b, 70a, 71, 73b, 74, 76, 77a, 77b, 78a, 79d, 79e, 79f, 80c, 81c, 81d, 82d, 83a, 86, 87a, 88a, 88e, 92b, 93d
bplist: 69c, 82a, 82d, 82e, 83a, 83b, 84a
Breakpoint: 81f, 95
breakpoint(): 73c, 82e
Breakpoint.addr: 81f
Breakpoint.count: 81f
Breakpoint.done: 81f
Breakpoint.next: 82b
Breakpoint.type: 81f
Breakpoint (typedef): 95
breakpoint_kind: 81e
BRK: 52
brkchk(): 83b, 84a, 84c, 84d
Bss: 15g, 20c, 46c, 55a
buildargv(): 70b, 73f
BY2PG: 18g, 19a, 19c, 19f, 20c, 20d, 45, 46a, 46b, 46c, 47c, 47d, 48e, 48f, 49a, 50b, 50c, 50d, 55a, 69c, 88a
BY2WD: 18g, 20e
calltree: 71, 81a, 81b, 81c, 81d
CARITH0: 25f
CARITH1: 25e, 25f, 29b
CARITH2: 25e, 25f, 30a
CARITH3: 32a, 32b, 32c
catcher(): 90a, 90b
CAUTO: 79e
CBLOC: 39b, 39c

CBRANCH: 40b, 40c
CCcmp: 28d, 28f, 34a, 34b, 41e, 42e, 42f
CCteq: 42a, 42e, 42f
CCtst: 42a, 42e, 42f
CDATA: 82d
CHDIR: 52
chdir(): 60c
CLOSE: 52
close(): 56a
cmd(): 17e, 21c
cmdcount: 67e, 68c, 82e, 85i
CMEM0: 34f, 35b, 36a
CMEM1: 35a, 35b, 36b
CMEM2: 35b, 35d
CMEM_BASIS: 34f, 35a, 35b, 35d
CMUL: 25c, 25d, 33b
colon(): 22a, 22b
compare_op: 42e
cond<>: 23c, 92d, 92d
count: 22c, 23a, 23c, 62b, 73f, 73g, 83b, 90b
CPARAM: 79f
CREATE: 52
create(): 59b
CTEXT: 73b, 74, 77a, 80c, 81c, 81d, 82d
Data: 15g, 19f, 46b, 55a
datasize: 20a, 20b, 55a
delbpt(): 73d, 83a
dobplist(): 71, 82d
dollar(): 68b, 71
dot: 21a, 21c, 22b, 68c, 69d, 70a, 73b, 74, 77a, 77b
dotlb(): 47b, 47c
dpex(): 27a, 27e, 29c, 30b, 32d
dumpdreg(): 71, 78c
dumpfreg(): 71, 78b
dumpreg(): 53a, 71, 78a
DUP: 52
dup(): 61b
emalloc(): 19c, 19f, 20c, 20d, 46a, 46b, 46c, 53c, 56b, 82e, 85g, 94a
Equal: 81e, 82d, 82e, 83b
erealloc(): 55a, 94b
errbuf: 54b, 54c, 55a, 55b, 56a, 56b, 57b, 58a, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62c, 62d
errjmp: 45, 47d, 50c, 50d, 53c, 93b, 93c, 93d
ERRMAX: 54b, 93a
ERRSTR: 52
errstr(): 55b, 56a, 56b, 57b, 58a, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62c, 62d
eval(): 68d, 76
EXEC: 52
EXITS: 52
expr(): 68c, 70a, 76, 77b, 82e, 83a

false: 18b, 18d, 22c, 27e, 28f, 29c, 36c, 43a, 43c, 43d, 45, 53c, 71, 73f, 73g, 86, 87a, 94a, 94b
FARM: 18b
fatal(): 17e, 18b, 18d, 45, 46a, 46b, 53c, 86, 93a, 94a, 94b
FAUTH: 52
FD2PATH: 52
fd2path(): 60b
fhdr: 18a, 18b, 18d, 19a, 19c, 19f, 20b
file: 17a, 17a, 17e, 20e
fmt: 68f, 68f, 73b, 76, 77a
FRAMENAME<>: 80b, 80c
FSTAT: 52
fstat(): 59a
FVERSION: 52
FWSTAT: 52
getmem_2(): 49c, 74
getmem_4(): 50a, 74, 79e, 79f, 80c, 83b
getmem_b(): 35h, 36c, 37b, 48e, 49c, 50a, 53c, 74
getmem_h(): 37b, 48f, 48f
getmem_v(): 49b, 57a, 57b, 58b
getmem_w(): 35h, 36c, 39e, 49a, 49a, 49b, 54c, 55a, 55b, 56a, 56b, 57b, 58a, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62b, 62c, 62d, 63b
Iarith: 14c, 25g, 26c, 29b, 30a, 32c, 33c, 86
Ib(): 41a, 41b
Ibl(): 41c, 41d
Ibranch: 14c, 41a, 41c, 86
Icache: 48a, 95
icache: 48b, 48c
Icache.hash: 48a
Icache.hashtext: 48a
Icache.lines: 48a
Icache.linesize: 48a
Icache.on: 48a
Icache.stall: 48a
Icache (typedef): 95
Idp0(): 26c, 27a
Idp1(): 29b, 29c
Idp2(): 30a, 30b
Idp3(): 32c, 32d
ifetch(): 23c, 47d
Ilsm(): 39d, 39e
Imem: 14c, 35g, 36a, 36b, 37a, 39d, 86
Imem1(): 36a, 36b, 36c
Imem2(): 37a, 37b
Imisc: 14c, 15c
Imul(): 25g, 26a
Imula(): 25g, 26b
Imull(): 33c, 33d
inc: 69b, 74, 77a
inithdr(): 17e, 18b

initmemory(): 17e, 19a
initstk(): 17e, 20e, 73f
Inst: 15a, 95
Inst.count: 85a
Inst.func: 15a
Inst.name: 15a
Inst.taken: 85a
Inst.type: 15a
Inst.useddelay: 85a
Inst (typedef): 95
Instruction: 81e, 82d, 82e, 83a, 84a
instruction (typedef): 14a
iprof: 47d, 69c, 85d, 85g, 88e
iprofile(): 71, 88e
isprint: 74
isum(): 71, 86
Iswap(): 35g, 35h
Isyscall: 14c, 44c, 86
itab: 15b, 15b, 23c, 85h, 86
itrace(): 23c, 53a, 54c, 55a, 55b, 56a, 56b, 57b, 58a, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62b, 62c, 62d, 63b, 79b, 92b
main<>(): 17e
membpt: 82e, 83a, 84b, 84c, 84d
memio(): 53c, 54c, 55b, 56b, 58a, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 62b
Memory: 16a, 95
memory: 16b, 19c, 19f, 20c, 20d, 45, 55a, 69c, 88a
Memory.seg: 16a
Memory (typedef): 95
MemRead: 53b, 53c, 58a, 62b
MemReadstring: 53b, 53c, 55b, 58c, 59b, 60a, 60c, 61a
MemWrite: 53b, 53c, 54c, 56b, 58c, 59a, 60b
MOUNT: 52
NCONT: 90b
NDFLT: 90b
nelem: 53a
nextc(): 22b, 67e, 68a, 70a, 71, 73g, 76, 77a, 77b, 82e
nil: 45, 53a, 53c, 67e, 94a, 94b
Nmaxtlb: 46d, 46e
nofunc: 63a, 63b
NOP: 52
nopcount: 85j, 86
NOTED: 52
noted(): 90b
NOTIFY: 52
notify(): 90a
Nseg: 15g, 16a, 45, 69c, 88a
numsym(): 69d, 70a
OADC: 24b, 26c, 29b, 30a, 32c, 34c
OADD: 24b, 26c, 28f, 29b, 30a, 32c

OAND: 24b, 26c, 28a, 29b, 30a, 32c
OB: 40a, 41a
OBIC: 24b, 26c, 28b, 29b, 30a, 32c
OBL: 40a, 41c
OCMN: 24b, 26c, 29b, 30a, 32c, 34a
OCMP: 24b, 26c, 29b, 30a, 32c, 41e
OEOR: 24b, 26c, 28a, 29b, 30a, 32c
OERRLEN<>: 62a, 62b
OLDB: 34e, 36a, 36b
OLDBU: 35c, 37a
OLDH: 35c, 37a
OLDM: 39a, 39d
OLDW: 34e, 36a, 36b
OMOV: 24b, 26c, 29b, 30a, 32c, 34d
OMUL: 25b, 25g
OMULA: 25b, 25g
OMULAL: 33a, 33c
OMULALU: 33a, 33c
OMULL: 33a, 33c
OMULLU: 33a, 33c
OMVN: 24b, 26c, 29b, 30a, 32c, 34d
OORR: 24b, 26c, 28a, 29b, 30a, 32c
opcode: 14b
opcode_category: 14c
OPEN: 52
open(): 17e, 55b
OREAD: 17e
ORSB: 24b, 26c, 29b, 30a, 32c, 34b
ORSC: 24b, 26c, 29b, 30a, 32c, 34c
OSBC: 24b, 26c, 29b, 30a, 32c, 34c
OSTB: 34e, 36a, 36b
OSTBU: 35c, 37a
OSTH: 35c, 37a
OSTM: 39a, 39d
OSTW: 34e, 36a, 36b
OSUB: 24b, 26c, 29b, 30a, 32c, 41e
OSWI: 44a, 44b, 44c
OSWPBU: 35e, 35f, 35g
OSWPW: 35e, 35f, 35g
OTEQ: 24b, 26c, 29b, 30a, 32c, 42a
OTST: 24b, 26c, 29b, 30a, 32c, 42a
OUNDEF: 14b, 15c, 24a, 44b
OWRITE: 17e
page_of_vaddr(): 45, 47d, 48e, 48f, 49a, 50b, 50c, 50d
Percent<>: 85c, 86, 87a
pfmt(): 74, 76, 77a
PIPE: 52
pipe(): 62d
PREAD: 52

pread(): 56b
printlocals(): 79e, 80c
printparams(): 79f, 80c, 81d
printsource(): 73b, 74, 79d, 80c, 81d, 88e
Prof: 88b, 101b
Prof.count: 88b
Prof.s: 88b
Prof (typedef): 101b
profcmp(): 88d, 88e
PROFGRAN: 47d, 85e, 85g, 88e
putmem_b(): 20e, 35h, 36c, 37b, 50b, 53c
putmem_h(): 37b, 50c
putmem_v(): 51, 57b
putmem_w(): 20e, 35h, 36c, 39e, 50d, 51, 62d
PWRITE: 52
pwrite(): 58a
quesie(): 68c, 77a
Read: 81e, 81e, 82d, 82e, 84c
reg: 15e, 19a, 20e, 21c, 22b, 23c, 26a, 26b, 27a, 27d, 28a, 28b, 28d, 28f, 29c, 30b, 30d, 31, 32d, 33d, 34a, 34b, 34d, 35h, 36c, 37b, 39e, 41b, 41d, 41e, 42a, 42c, 42f, 43a, 43c, 43d, 53a, 54a, 54c, 55a, 55b, 56a, 56b, 57a, 57b, 58a, 58b, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62b, 62c, 62d, 63b, 63c, 63d, 63e, 64a, 64b, 64c, 64d, 64e, 64f, 65a, 65b, 65c, 65d, 65e, 65f, 66a, 66b, 69c, 77b, 78a, 80c, 81c, 81d, 84a, 85b, 92b, 93d
REGARG: 15f, 53a, 63c, 63d, 63e, 64a, 64b, 64c, 64d, 64e, 64f, 65a, 65b, 65c, 65d, 65e, 65f, 66a, 66b
Registers: 15d, 95
Registers.ar: 23b
Registers.cbit: 28e
Registers.cc1: 42g
Registers.cc2: 42g
Registers.compare_op: 42d
Registers.cout: 28e
Registers.instr: 23b
Registers.instr_cond: 42b
Registers.instr_opcode: 23b
Registers.ip: 23b
Registers.r: 15d
Registers (typedef): 95
REGLINK: 15f, 41d, 80c
REGPC: 15f, 19a, 21c, 22b, 23c, 26a, 26b, 27b, 27c, 27d, 30c, 33d, 36c, 37b, 41b, 41d, 77b, 78a, 80c, 81c, 81d, 84a, 93d
REGRET: 15f, 54c, 55a, 55b, 56a, 56b, 58a, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62c, 62d, 63b, 81c
REGSP: 15f, 20e, 54c, 55a, 55b, 56a, 56b, 57a, 57b, 58a, 58b, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62b, 62c, 62d, 63b, 77b, 78a, 80c, 81d
REMOVE: 52
remove(): 60a
RENDEZVOUS: 52
reset(): 69c, 73f
RFORK: 52
run(): 22c, 23c, 73f, 73g
runcmp(): 42f, 43a

runteq(): 42f, 43c
runtst(): 42f, 43d
Sbit: 28c, 28d, 28f, 34a, 34b, 41e, 42a
SEEK: 52
seek(): 18b, 46a, 46b, 57b
SEEK_START: 18b
SEGATTACH: 52
SEGBRK: 52
SEGDETACH: 52
SEGFLUSH: 52
SEGFREE: 52
Segment: 16c, 95
Segment.base: 16c
Segment.end: 16c
Segment.fileend: 16c
Segment.fileoff: 16c
Segment.refs: 87c
Segment.rss: 87c
Segment.table: 16c
Segment.type: 16c
Segment (typedef): 95
segment_kind: 15g
segsum(): 71, 88a
setreg(): 68e, 77b
shift(): 29c, 30b, 30d, 36c
shtype<>: 92c, 92c
SIGNBIT: 43b, 43c, 43d
SLEEP: 52
sleep(): 62c
Ssyscall(): 44c, 53a
Stack: 15g, 20d, 46c, 55a
STACKSIZE: 18g, 20d
STACKTOP: 18g, 20d, 20e
STARTSYM<>: 80a, 80c
STAT: 52
stat(): 58c
STATMAX: 58c, 59a
STDERR: 93a
STDIN: 17e
STDOUT: 17e
stktrace(): 71, 80c
STRINGSZ<>: 79c, 79d
stype: 87b, 87b, 88a
symmap: 18c, 18d, 74
sysalarm(): 52, 65f
sysawait(): 52, 63e
sysbind(): 52, 61a
sysbrk(): 52, 55a
syschdir(): 52, 60c

sysclose(): 52, 56a
syscreate(): 52, 59b
sysctab: 53a, 54a, 63c, 63d, 63e, 64a, 64b, 64c, 64d, 64e, 64f, 65a, 65b, 65c, 65d, 65e, 65f, 66a, 66b
sysdbg: 54c, 55a, 55b, 56a, 56b, 57b, 58a, 58c, 59a, 59b, 60a, 60b, 60c, 61a, 61b, 62b, 62c, 62d, 63b, 71, 79a, 79b
sysdup(): 52, 61b
syserrstr(): 52, 54c
sysexec(): 52, 63d
sysexits(): 52, 62b
sysfauth(): 52, 66a
sysfd2path(): 52, 60b
sysfstat(): 52, 59a
sysfversion(): 52, 66b
sysfwstat(): 52, 64b
sysmount(): 52, 65d
sysnop(): 52, 54a
sysnoted(): 52, 64c
sysnotify(): 52, 63b
sysopen(): 52, 55b
syspipe(): 52, 62d
syspread(): 52, 57a
syspwrite(): 52, 58b
sysread(): 56b, 57a
sysremove(): 52, 60a
sysrendezvous(): 52, 65c
sysrfork(): 52, 63c
sysseek(): 52, 57b
syssegattach(): 52, 64d
syssegbrk(): 52, 65b
syssegdetach(): 52, 64e
syssegflush(): 52, 65a
syssegfree(): 52, 64f
syssleep(): 52, 62c
sysstat(): 52, 58c
systab: 52, 52, 53a
sysunmount(): 52, 65e
syswrite(): 58a, 58b
syswstat(): 52, 64a
tables: 85h, 85h, 86
Text: 15g, 19c, 46a
text: 17b, 17e, 46a, 46b
textbase: 19d, 19e, 47d, 88e
Tlb: 46d, 95
tlb: 46f, 47a, 47b, 47c, 87a
Tlb.hit: 46d
Tlb.miss: 46d
Tlb.on: 46d
Tlb.tlbent: 46d
Tlb.tlbsize: 46d

Tlb (typedef): 95
tlbsum(): 71, 87a
trace: 23c, 53a, 71, 92a
true: 17e, 28a, 28b, 28f, 30b, 34d, 43a, 43c, 43d, 46a, 46b, 47a, 71, 81b, 83b
undef(): 15c, 26a, 26b, 33d, 34c, 39e, 43a, 43c, 43d, 93d
UNMOUNT: 52
updateicache(): 48c, 48d
UTZERO: 18g
width: 69a, 69a, 74, 77a
Write: 81e, 81e, 82d, 82e, 84d
WSTAT: 52
__anon_enum_12: 15f
__anon_enum_13: 53b
__anon_enum_14: 18g

Bibliography

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 9

[Knu99] Donald E. Knuth. *MMIXWare, a RISC Computer for the Third Millenium*. Springer, 1999. cited page(s) 8

[KR88] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. cited page(s) 9

[NS05] Noam Nisan and Shimon Shocken. *The Elements of Computing Systems*. MIT Press, 2005. cited page(s) 6, 7, 8

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 9

[PH13] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 2013. cited page(s) 7, 9

[Tan88] Andrew S Tanenbaum. *Structured Computer Organization*. Prentice Hall, 1988. cited page(s) 9